



中南大学

AE 开发期末设计报告

项目名称：_____ 城市交通系统 _____

指导老师：_____

组 长 : _____

组 员 : _____

撰写人 : _____

中南大学地球科学与信息物理学院

2024 年 12 月

目录

1 项目概况.....	4
1.1 引言.....	4
1.1.1 选题背景:	4
1.2 术语定义.....	4
1.3 技术路线.....	5
1.3.1 技术选型:	5
1.3.2 总体技术架构设计.....	5
1.4 需求分析.....	6
1.4.1 基础功能需求.....	6
1.4.2 空间分析需求.....	6
1.4.3 数据需求性分析.....	7
1.4.4 其他需求.....	7
2 开发任务.....	8
2.1 用户界面开发.....	8
2.1.1 任务描述.....	8
2.1.2 技术要点.....	8
2.1.3 交付成果.....	8
2.2 基础地理信息操作功能实现.....	8
2.2.1 要素编辑功能:	8
2.2.2 要素删除功能.....	9
2.2.3 属性表功能.....	9
2.2.4 要素类创建.....	9
2.2.5 鹰眼地图功能.....	10
2.2.6 栅格渲染.....	10
2.3 空间查询.....	11
2.3.1 按属性查找.....	11
2.3.2 交通站点查询:	11
2.3.3 统计图表.....	12
2.4 空间分析计算.....	13
2.4.1 噪音分析.....	13
2.4.2 栅格叠加计算.....	13
2.4.3 最短路径分析功能.....	13
3 数据库介绍.....	14
3.1 用户信息数据库.....	14
3.1.1 本体.....	14
3.1.2 用户表:.....	14
3.2 地理文件型数据库 (gdb)	14
3.2.1 成都市.gdb (只列出空间分析主要使用要素类):.....	14
3.3 路网.gdb.....	15
4 程序编码.....	16
4.1 开发环境.....	16
4.2 功能代码.....	16

4.2.1 属性表操作.....	16
4.2.2 公交站点查询:	23
4.2.3 噪音分析.....	25
4.2.4 栅格渲染.....	29
4.2.5 最优路径分析.....	32
5 系统运行.....	35
5.1 系统运行环境配置.....	35
5.1.1 Setup 配置.....	35
5.2 登录功能.....	36
5.2.1 界面演示.....	36
5.2.2 操作流程.....	36
5.3 基础地理信息操作.....	36
5.3.1 管理员系统界面.....	36
5.3.2 打开地图文档 (mxd)	37
5.3.3 属性表浏览.....	37
5.4 空间查询.....	39
5.4.1 按属性查询.....	39
5.4.2 空间统计.....	40
5.4.3 交通站点查询.....	42
5.5 要素类创建与要素编辑.....	43
5.5.1 创建要素类.....	43
5.5.2 矢量编辑器.....	45
5.5.3 属性表编辑.....	45
5.5.4 地图选择要素删除.....	46
5.5.5 属性表批量删除要素.....	48
5.6 空间分析.....	49
5.6.1 噪音计算 (线密度分析)	49
5.6.2 叠加分析 (地图代数)	50
5.6.3 栅格渲染.....	51
5.7 路径分析.....	52
5.7.1 最短路径生成.....	52
6 项目总结.....	54
6.1 系统测试.....	54
6.2 项目总结.....	54
7 附件.....	55
7.1 操作演示视频.....	55
7.2 项目源码.....	55
7.3 地图数据 (不含卫星地图)	55
7.4 远程仓库链接:	55

1 项目概况

1.1 引言

1.1.1 选题背景:

随着城市化进程的飞速推进，城市交通系统作为城市运行的关键脉络，其复杂程度与重要性与日俱增。在城市居民生活品质日益受到关注的当下，交通系统所产生的噪音污染问题愈发凸显，成为不可忽视的环境负面因素。与此同时，高效的交通出行需求也促使人们对最短路径规划提出了更高的精准性与及时性要求。

因此，对城市交通系统进行二次开发，整合先进的算法与实时数据，优化最短路径计算功能，能够使交通参与者更高效地规划行程，减少出行时间与成本，提高城市整体运行效率，在缓解交通拥堵、节能减排以及提升公共服务响应速度等方面都具有显著的现实意义，为打造绿色、智能、宜居的现代化城市交通环境奠定坚实基础。

1.2 术语定义

术语	定义
GIS	地理信息系统（GIS）是一种用于采集、存储、分析、管理和展示地理空间数据的计算机系统，它能够将地理空间数据与属性数据相结合，为资源管理、城市规划、环境保护等众多领域提供决策支持。
空间查询	空间查询是地理信息系统（GIS）中的一种操作方式，它允许用户根据地理要素的空间位置（如点、线、面之间的包含、相交、临近等关系）和属性信息，从空间数据库中快速检索出符合特定条件的地理要素及其相关数据。
叠加分析	叠加分析是地理信息系统（GIS）中的一种空间分析方法，通过将两个或多个不同图层的地理数据按照一定规则（如相交、合并、相减等）进行组合运算，从而产生新的空间信息，用于揭示要素之间的空间关系和综合特征。
线密度分析	线密度分析是 GIS（地理信息系统）中的一种空间分析方法，它用于计算单位面积内线性要素（如道路、河流等）的长度，以此来反映线性要素在空间分布的密集程度。

网络分析	网络分析是地理信息系统（GIS）中的一种重要技术，主要用于研究交通网络、水电气管道网络等各类网络系统中资源的流动、路径的选择、设施的布局以及连通性等问题。
------	---

表 1 术语和定义

1.3 技术路线

1.3.1 技术选型：

配置选项	内容
开发语言	Microsoft Visual C# .NET
集成开发环境（IDE）	Microsoft Visual 2019
基础地理信息系统环境	ArcGIS Engine 10.8
用户组件库	DevExpress 20.1
数据库	SQL Server Management Studio 19

表 2 技术选型

1.3.2 总体技术架构设计

本桌面应用采用经典的三层架构模式，包括表示层、业务逻辑层和数据访问层，各层之间通过接口进行松耦合通信，以提高系统的可维护性和可扩展性：

1.3.2.1 表示层：

使用 C# 和 Windows Forms 技术结合 DevExpress 控件构建用户界面，负责与用户进行交互，接收用户输入并展示系统处理结果。通过事件驱动机制响应用户操作，如按钮点击、菜单选择等，并将用户请求传递给业务逻辑层进行处理。

在界面设计上，遵循简洁直观、易于操作的原则，利用 DevExpress 的主题和样式定制功能，确保应用程序在不同操作系统和分辨率下都具有一致且美观的外观。同时，注重用户操作习惯和交互流程的优化，提高用户工作效率。

1.3.2.2 业务逻辑层：

封装了应用程序的核心业务逻辑，协调表示层和数据访问层之间的交互，对来自表示层的请求进行业务规则验证、业务流程处理和数据处理逻辑实现。例如，在地理信息处理方面，负责组织和调用 ArcEngine 提供的各种 GIS 功能组件，实现地图的加载与显示控制、地理要素的查询与分析、空间数据的编辑与更新等业务操作，并将处理结果返回给表示层进行展示。

采用面向对象的设计方法，将不同的业务功能模块封装成独立的类和方法，提高

代码的复用性和可维护性。同时，通过接口定义业务逻辑层与其他层之间的交互规范，便于进行单元测试和代码维护。

1.3.2.3 数据访问层：

负责与数据库和地理空间数据文件进行交互，实现数据的读取、写入、查询和更新操作。对于关系型数据库，使用 ADO.NET 技术进行数据访问，封装数据库连接、命令执行、数据读取等操作，确保数据的高效访问和安全性。对于地理空间数据，通过 ArcEngine 的数据访问接口，实现对各种地理数据格式（如 Shapefile、Geodatabase 等）的读取和写入操作，为业务逻辑层提供地理数据支持。

1.4 需求分析

1.4.1 基础功能需求

1.4.1.1 要素类创建：

业务上面体现为实现路点规划（比如新建立一种全新的交通方式）使用 ArcEngine 的地理数据库操作接口，根据城市交通系统的需求创建各种要素类。例如，对于道路要素类，需要创建线要素类，包含道路名称、道路等级、车道数、限速等属性。

1.4.1.2 要素编辑

通过用户界面或数据导入功能，向要素类中添加新的要素。例如，对于道路要素类，用户可以在地图上绘制新的道路，将其添加到道路要素类中；对于公交站点要素类，添加新的站点位置信息。

1.4.1.3 要素修改

提供用户界面（属性表），允许用户修改现有要素的属性信息。例如，修改道路的名称。

1.4.1.4 要素删除

允许用户删除不再需要的要素，如废弃的道路、拆除的公交站点等。实现批量删除功能（属性表操作），提高操作效率，例如一次删除多个废弃的交通设施要素。

1.4.1.5 要素查询

制作按属性查找的 SQL 计算器，利用 SQL 语句实现空间查询。

1.4.2 空间分析需求

1.4.2.1 线密度分析：

使用 ArcEngine 开发一个城市交通噪音分析模块，能够基于道路要素类中自带的噪音字段，通过线密度分析生成噪音栅格图，为城市交通噪音的空间分布评估和可视化提供支持。

1.4.2.2 栅格叠加分析：

将不同道路的噪音栅格图进行叠加分析（地图代数运算）可全面评估城市噪音水

平、优化交通规划、评价环境影响及评估降噪措施效果，为城市交通和环境管理提供综合决策依据。

1.4.2.3 最短路径分析：

在城市交通系统中，最短路径功能结合自带网络数据集、考虑时间成本并输出导航信息至关重要。对于交通参与者而言，它能够根据网络数据集中的道路连接性、通行限制等信息，以及时间成本因素（如不同路段的车速限制、交通拥堵状况等），为用户规划出最省时的出行路线，极大提高出行效率，减少出行时间成本。

1.4.3 数据需求性分析

1.4.3.1 城市基础地理数据需求

城市基础地理信息需求包括精确的地形地貌数据、道路网络信息、土地利用情况以及各类地理实体（如建筑物、水系、绿地等）的空间分布和属性信息，为城市交通系统的开发和优化提供基础支撑。

1.4.3.2 城市需求分析

网络数据集需求包括清晰准确的道路节点及其唯一标识、路段的详细几何信息和连通性描述、限速等静态属性，以及实时更新的交通流量、路况等动态数据，为实现最短路径规划和时间成本计算提供完整的数据基础。

1.4.4 其他需求

1.4.4.1 系统兼容性需求：

软件需要兼容主流的操作系统，如 Windows。以满足不同用户群体和使用场景的需求。对于 Windows 系统，需要支持不同的版本，包括 Windows 10 及以上，因为它是大多数办公和个人用户的常用操作系统，具有广泛的用户基础

操作系统的兼容性保证了软件可以在各种硬件平台上顺利安装和运行，同时也方便与其他基于不同操作系统的系统进行集成，如与交通监控系统、交通数据服务器等的对接。

1.4.4.2 开发框架和运行环境需求：

对于使用 ArcEngine 进行开发的部分，需要安装相应的 ArcGIS 运行时环境，以确保软件能够调用 ArcEngine 的功能模块进行地理信息处理和空间分析。同时，需要使用 .NET Framework 或 Java 等开发框架，这些框架提供了强大的开发工具和丰富的类库，方便开发人员进行软件开发，提高开发效率和软件的稳定性。

1.4.4.3 硬件要求

为了处理大量的地理信息数据、执行复杂的空间分析（如噪音分析、最短路径计算）和实现流畅的用户界面交互，软件需要一定性能的处理器的支持。对于普通用户端，至少需要多核处理器，如 Intel Core i5 或 AMD Ryzen 5 及以上，以确保能够快速响应用户操作，如地图的加载、缩放、路径规划等。

考虑到需要加载和处理的地理信息数据量较大，如城市的道路网络、噪音数据、地形数据等，软件在运行时需要足够的内存支持。普通用户端至少需要 8GB 的内存，以保证在打开地图和进行简单分析时不会出现卡顿现象；服务器端根据服务的用户数

量和数据处理量，可能需要 32GB 甚至更高的内存，以满足多用户并发访问和大数据量处理的需求。

2 开发任务

2.1 用户界面开发

2.1.1 任务描述

设计并开发应用的登录界面，包括创建用户界面布局，使用 DevExpress 相关控件（如文本框用于输入账号密码、按钮用于登录操作等）构建美观且易用的界面展示形式。实现与后端 SQL Server 数据库的连接，以验证用户输入的账户信息的准确性，确保只有合法用户能够成功登录系统，同时提供友好的错误提示信息，如账号不存在、密码错误等情况的反馈。并根据用户身份的不同进入不同的系统。

2.1.2 技术要点

利用 C# 的 Windows Forms 技术结合 DevExpress 控件进行界面设计，通过 ADO.NET 实现与 SQL Server 数据库的交互，包括数据库连接的建立、查询语句的执行以及结果的处理，确保登录验证过程的高效性和安全性。

2.1.3 交付成果

可正常运行的登录界面模块，实现准确的用户身份验证功能，并与系统其他模块无缝集成，确保用户登录后能够顺利进入主应用程序界面。

2.2 基础地理信息操作功能实现

2.2.1 要素编辑功能：

2.2.1.1 任务描述

在地图数据加载后，允许用户对地理要素进行编辑操作。主要通过属性表的操作来实现。通过 ArcEngine 的编辑工具和接口，提供直观的编辑操作界面，支持撤销和重做功能，以保证用户在编辑过程中的操作灵活性和数据准确性，编辑结果能够实时更新并保存到地理数据库中。

2.2.1.2 技术要点

运用 ArcEngine 的编辑功能组件，如 IEditor 接口和相关的编辑工具类，实现对要素几何和属性的编辑操作。通过记录编辑操作的历史记录，利用堆栈数据结构实现撤销和重做功能，确保编辑过程的可回溯性和数据的一致性。在数据保存方面，使用 DevExpress 的 gridControl 操作将修改后的要素数据提交到数据库中，保证数据的持久化存储。

2.2.1.3 交付成果

具备完整要素编辑功能的模块，能够满足用户对地理要素的各种编辑需求，确保编辑后的要素数据准确无误地保存到地理数据库中，并在地图上实时显示更新后的要素状态。

2.2.2 要素删除功能

2.2.2.1 任务描述

用户能够选择并删除地图上特定地理要素的功能。用户可以通过鼠标点击或框选等方式选择要删除的要素。也可以使用属性表操作来实现。

2.2.2.2 技术要点

用 ArcEngine 的选择工具和数据访问接口，实现对用户选择要素的识别和获取，通过执行数据库删除操作

2.2.2.3 交付成果

稳定可靠的要素删除功能模块，能够准确地删除用户指定的地理要素，保证地图数据和数据库的一致性。

2.2.3 属性表功能

2.2.3.1 任务描述

实现用户在地图操作过程中能够方便地打开地理要素的属性表，以查看和编辑要素的详细属性信息。属性表应以直观、清晰的表格形式展示要素的各个属性字段及其对应的值，支持对属性数据的排序、筛选和编辑操作，用户在属性表中进行的修改能够实时同步到地图上的相应要素，并保存到数据库中。

2.2.3.2 技术要点

使用 ArcEngine 的属性表组件（如 ITable 接口）来创建和显示属性表，通过数据绑定技术将属性表与地理要素数据集进行关联，实现属性数据的实时更新和交互操作。在编辑属性数据时，结合前面提到的要素编辑功能，确保对属性的修改能够正确地反映在地理要素上，并通过数据库事务处理机制保证数据的完整性和一致性，避免因部分数据更新失败而导致的数据不一致问题。

2.2.3.3 交付成果

功能完善的属性表打开与编辑功能模块，能够快速、准确地展示和编辑地理要素的属性信息，与地图操作和数据库存储紧密集成，为用户提供便捷、高效的属性数据管理工具。

2.2.4 要素类创建

2.2.4.1 任务描述

允许用户在地理数据库中创建新的要素类，定义要素类的几何类型。性字段（如名称、面积、长度等）。通过友好的用户界面引导用户进行要素类的创建操作，确保

创建的要素类符合地理数据库的规范和用户的实际需求，创建完成后能够在地图中进行显示和进一步的编辑操作。

2.2.4.2 技术要点

使用 ArcEngine 的工作空间创建工具和要素类定义接口，在地理数据库（如 Geodatabase）中创建新的要素类对象，并根据用户输入的信息设置要素类的属性和几何定义。在创建过程中，进行数据合法性验证和默认值设置，确保要素类的完整性和可用性

2.2.4.3 交付成果

能够实现要素类创建功能的模块，用户可以通过简单的操作在地理数据库中创建符合需求的要素类，并能够在地图中对其进行可视化和编辑操作，为地理数据的创建和管理提供便捷的途径。

2.2.5 鹰眼地图功能

2.2.5.1 任务描述

创建鹰眼地图功能，以提供地图的整体概览和快速定位功能。在主地图窗口旁边显示一个小的鹰眼地图窗口，其中显示整个地图的缩略图，并在缩略图上用矩形框标识出当前主地图窗口显示的区域范围。用户可以通过在鹰眼地图上拉动矩形框来快速切换主地图的显示范围，实现对地图的快速浏览和定位操作，提高用户在处理大区域地理数据时的操作效率和便捷性。

2.2.5.2 技术要点

利用 ArcEngine 的地图显示和图形绘制功能，在独立的窗口中创建鹰眼地图的缩略图显示，并通过监听主地图的显示范围变化事件，实时更新鹰眼地图上的矩形框位置和大小。同时，在鹰眼地图上添加鼠标交互事件处理，实现通过鼠标操作来控制主地图的显示范围切换，确保主地图和鹰眼地图之间的交互流畅性和准确性，为用户提供直观、便捷的地图浏览体验。

2.2.5.3 交付结果

功能完善的鹰眼地图模块，能够与主地图紧密配合，提供地图的整体概览和快速定位功能，提高用户对地理数据的浏览和操作效率，增强应用程序的易用性和实用性。

2.2.6 栅格渲染

2.2.6.1 任务描述

对于拉伸渲染，需根据栅格数据的值域范围，通过特定的数学算法将数据值映射到色彩空间，使得数据的变化能够以颜色的连续过渡直观呈现，突出数据的细节特征和整体分布趋势。在分级渲染方面，依据预先设定的分级标准（如等间距、分位数等）将栅格数据值划分为不同的级别区间，为每个级别区间赋予独特的颜色或符号，从而清晰地展示数据的分布层次和类别差异

2.2.6.2 技术要点

在拉伸渲染中，首先对栅格数据进行统计分析，获取其最小值、最大值等关键统计量，基于这些统计量选择合适的拉伸方法（如线性拉伸、标准差拉伸等），通过构建数学函数将栅格数据值转换为 0 - 255 的颜色索引值，进而映射到选定的色彩映射表（如灰度、彩虹色带等）上，实现颜色的渐变分配，确保颜色过渡自然且能准确反映数据变化。对于分级渲染，根据数据特点和分析需求确定分级方式和级数，例如对于均匀分布的数据可采用等间距分级，对于偏态分布的数据则采用分位数分级更能突出数据特征。通过编写代码或利用库函数创建分级区间，并为每个区间指定相应的颜色或符号，将栅格数据的每个像元按照其数值所属的分级区间进行渲染，同时提供图例生成功能，清晰标注每个分级区间的数值范围和对应的颜色或符号

2.2.6.3 交付结果

能够对输入的栅格数据准确地执行拉伸渲染和分级渲染操作，用户可以通过简洁的界面交互（如菜单选择、参数设置对话框等）灵活切换渲染方式、调整拉伸参数和分级标准，并实时预览渲染效果

2.3 空间查询

2.3.1 按属性查找

2.3.1.1 任务描述

实现用户根据地理要素的属性信息进行查找和筛选的功能。用户可以通过输入特定的属性条件（如某个字段的值等于特定值、包含特定字符串等），系统在地图上快速定位并突出显示符合条件的地理要素，同时在属性表中筛选出相应的记录，方便用户对特定要素进行查看和进一步的操作。支持多种属性条件的组合查询，以满足用户复杂的查询需求，提高地理数据的查询效率和准确性。

2.3.1.2 技术要点

使用 ArcEngine 的查询过滤器（QueryFilter）和选择集（ISelectionSet）功能，根据用户输入的属性条件构建查询表达式，对地理要素数据集进行筛选和查询操作。通过设置地图图层的选择符号化方式，将符合条件的要素在地图上以突出的方式显示。

2.3.1.3 交付结果：

高效、灵活的按属性查找功能模块，能够准确地根据用户输入的属性条件查找和定位地理要素，在地图和属性表中提供直观的查询结果展示，为用户提供便捷的地理数据查询工具，提高数据处理和分析的效率。

2.3.2 交通站点查询：

2.3.2.1 任务描述

在桌面应用程序的主界面中，设置一个明显且易于操作的搜索框，用户在此输入公交或地铁站点的名称（支持部分名称模糊查询），输入完成后点击搜索按钮或者按

下回车键发起查询指令。

系统接收到查询请求后，迅速在后台数据库中检索匹配的公交和地铁站点信息，数据库中存储了站点的名称、坐标位置、所属线路等详细数据，这些数据与地图图层中的地理要素精确关联。

一旦找到目标站点，地图将自动以平滑过渡的方式缩放至该站点所在区域，以合适的比例尺展示站点周边的地理环境，同时，目标站点要素在地图上通过闪烁、变色或放大等动态效果进行突出显示

2.3.2.2 技术要点

利用 SQL 语句查询，基于 ArcEngine 实现地图缩放和平移及站点闪亮效果，同时注重搜索框等用户界面交互设计。

2.3.2.3 交付成果

一个集成了公交和地铁站点查询功能的桌面应用程序模块，用户能够通过简洁的搜索操作，快速准确地在地图上定位到目标站点，并通过直观的闪亮效果突出显示查询结果，满足用户对公交和地铁站点信息查询和地图可视化的需求。

2.3.3 统计图表

2.3.3.1 任务描述

本任务旨在创建一个数据可视化模块，能够从预先准备好的数据源（如地理信息数据库或交通数据文件）中提取关于道路限速和类型的相关数据，并将其转化为直观易懂的统计图表。具体而言，针对各个区间限速路段，系统将统计每个路段的限速数值，并以柱状图的形式展示不同路段限速的分布情况，柱子的高度代表限速值，x 轴标注路段区间，使限速差异一目了然。对于不同道路类型的占比，通过计算各类道路（如高速公路、城市主干道、乡村小道等）在总道路里程或数量中的比例，以饼状图呈现，直观地展示各道路类型的相对占比关系，每个扇形区域代表一种道路类型及其占比份额。而不同道路类型的限速情况，则利用雷达图进行可视化，雷达图的每个轴表示一种道路类型，轴上的刻度为限速数值，通过绘制多边形来清晰呈现不同道路类型限速的对比情况，方便用户快速把握各类道路限速的整体特征和差异。

2.3.3.2 技术要点

从数据源中精准提取所需的道路限速和类型信息，并进行必要的数据库清洗和预处理，确保数据的准确性和完整性。在图表绘制过程中，借助专业的图表绘制库 DevExpress 图表控件，根据不同图表类型（柱状图、饼状图、雷达图）的特点和需求，对数据进行合理的组织和布局。通过设置图表的标题、坐标轴标签、颜色、图例等属性，增强图表的可读性和美观性。

2.3.3.3 交付结果

最终交付的成果包括一个功能完备的统计图表生成模块，能够无缝集成到相关的交通数据分析或地理信息系统应用程序中。用户可以通过简单的操作指令（如菜单选择、按钮点击等）触发图表生成过程，系统将迅速从数据源获取最新数据并绘制出准确、清晰、美观的各个区间限速路段柱状图、不同道路类型占比饼状图以及不同道路类型限速雷达图，并在应用程序的界面中进行展示。

2.4 空间分析计算

2.4.1 噪音分析

2.4.1.1 任务描述

本任务聚焦于对地理数据集中的噪音字段进行线密度分析。计算单位长度上噪音的分布情况，生成直观栅格图呈现线密度分析结果。用户需要自己输入搜索半径和像元大小。

2.4.1.2 技术要点

合理设置分析参数，如搜索半径、像元大小等，以平衡分析结果的准确性和计算效率。利用空间分析工具将计算得到的线密度值转化为可视化的栅格图

2.4.1.3 交付结果

最终生成噪声的栅格影像图，以便后续进行叠加分析。

2.4.2 栅格叠加计算

2.4.2.1 任务描述

实现多个栅格数据图层的叠加分析功能，用于综合分析不同地理要素在空间上的相互关系和影响。用户可以选择两个或多个具有相同地理范围和分辨率的栅格数据图（层），系统将这些图层进行叠加运算（如加法、减法、乘法、除法等数学运算或逻辑运算），生成新的叠加结果栅格数据，并在地图上显示叠加后的效果，同时提供对叠加结果的统计分析和可视化展示功能，帮助用户深入了解地理要素之间的空间分布规律和相互作用关系。

2.4.2.2 技术要点

用 ArcEngine 的栅格数据处理和分析功能，实现栅格图层的叠加运算。首先，确保参与叠加的栅格数据图层具有相同的地理范围，调用事先写好的栅格叠加的 python 脚本，实现栅格与常数的运算。

2.4.2.3 交付结果

能够准确地对多个栅格数据图层进行叠加分析，生成具有实际应用价值的叠加结果数据，并提供丰富的可视化和统计分析功能。

2.4.3 最短路径分析功能

2.4.3.1 任务描述

开发最短路径分析功能，用于在地理网络数据（如道路网络、交通线路等）上计算两点之间的最短路径或最优路径。用户可以在地图上指定起始点和终点，系统根据网络数据集和相关的路径分析算法。计算出最短路径，并在地图上以突出的线条形式显示路径轨迹。同时输出导航信息在 listbox 里。

2.4.3.2 技术要点

利用 ArcEngine 的网络分析扩展模块（如 ESRI.ArcGIS.Network Analyst 工具集），构建地理网络数据集，并基于选定的路径分析算法实现最短路径的计

算。

2.4.3.3 交付结果

高效、准确的最短路径分析功能模块，能够根据用户指定的起始点和终点在地理网络数据上快速计算出最短路径，并提供详细的路径信息和可视化展示。

3 数据库介绍

3.1 用户信息数据库

3.1.1 本体

用 SQL Server 作为关系型数据库管理系统，用于存储和管理登录界面的用户信息。SQL Server 以其强大的功能、可靠性和安全性，为用户数据的持久化提供了稳定的基础架构，能够满足企业级应用对于数据存储、查询、更新和管理的严格要求。

3.1.2 用户表：

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	userID	Int		主码，自增
2	用户名	userName	Varchar	50	非空
3	密码	passwords	Varchar	50	非空
4	用户类型	userType	Varchar	50	非空

3.2 地理文件型数据库（gdb）

3.2.1 成都市.gdb（只列出空间分析主要使用要素类）：

3.2.1.1 道路：

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码，自增
2	几何形状	Shape	GEOMETRY		非空
3	道路名字	Name	Varchar	50	非空
4	道路类型	fclass	Varchar	50	非空
5	几何长度	Shape_Length	Double		非空
6	噪音	nosie	Double		
7	时间成本	time	Double		

3.2.1.2 火车轨道

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码，自增
2	几何形状	Shape	GEOMETRY		非空

3	轨道名字	Name	Varchar	50	非空
4	轨道类型	fclass	Varchar	50	非空
5	几何长度	Shape_Length	Double		非空
6	噪音	nosie	Double		

3.2.1.3 公交车站

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码, 自增
2	几何形状	Shape	GEOMETRY		非空
3	站点名字	Name	Varchar	50	非空
4	公交线路	line	Varchar	50	非空

3.2.1.4 地铁站

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码, 自增
2	几何形状	Shape	GEOMETRY		非空
3	站点名字	Name	Varchar	50	非空
4	地铁线路	line	Varchar	50	非空

3.3 路网.gdb

3.3.1.1 道路网:

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码, 自增
2	几何形状	Shape	GEOMETRY		非空
3	道路名字	Name	Varchar	50	非空
4	道路类型	fclass	Varchar	50	非空
5	几何长度	Shape_Length	Double		非空
6	噪音	nosie	Double		
7	时间成本	time	Double		非空

3.3.1.2 路网_ND_Junctions

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码, 自增
2	几何形状	Shape	GEOMETRY		非空

3.3.1.3 Stops (求最短路径时候, 存储用户起止点)

序号	字段说明/ 别名	字段名称	字段类型	长度	约束
1	标识码	OBJECTID	Int		主码, 自增
2	几何形状	Shape	GEOMETRY		非空

4 程序编码

4.1 开发环境

配置选项	内容
开发语言	Microsoft Visual C# .NET
集成开发环境（IDE）	Microsoft Visual 2010
基础地理信息系统环境	ArcGIS Engine 10.2
用户组件库	DevExpress 20.1
数据库	SQL Server Management Studio 19

4.2 功能代码

4.2.1 属性表操作

4.2.1.1 属性表与 gridView 的绑定

```
private void ProcessTableDataBasedOnLayerSelection()
{
    int selectedIndex = cbxLayerSelect.SelectedIndex;
    gridControl1.DataSource = null;
    if (selectedIndex >= 0)
    {
        ILayer pLayer = m_hookHelper.FocusMap.get_Layer(selectedIndex);
        IFeatureLayer pFLayer = pLayer as IFeatureLayer;
        IFeatureClass pFeatureClass = pFLayer.FeatureClass;

        if (pFeatureClass == null) return;

        System.Data.DataTable dt = new System.Data.DataTable();
        DataColumn dc = null;

        for (int i = 0; i < pFeatureClass.Fields.FieldCount; i++)
        {
            dc = new DataColumn(pFeatureClass.Fields.get_Field(i).Name);

            dt.Columns.Add(dc);
        }

        IFeatureCursor pFeatureCuror = pFeatureClass.Search(null, false);
```

```

        IFeature pFeature = pFeatureCuror.NextFeature();

        DataRow dr = null;
        while (pFeature != null)
        {
            dr = dt.NewRow();
            for (int j = 0; j < pFeatureClass.Fields.FieldCount; j++)
            {
                if (pFeatureClass.FindField(pFeatureClass.ShapeFieldName) == j)
                {

                    dr[j] = pFeatureClass.ShapeType.ToString();
                }
                else
                {
                    dr[j] = pFeature.get_Value(j).ToString();
                }
            }

            dt.Rows.Add(dr);
            pFeature = pFeatureCuror.NextFeature();
        }
        gridControl1.DataSource = dt;
        dt2 = dt;
    }
}

```

4. 2. 1. 2 属性表编辑

```

// 表格单元格值改变事件处理函数，将修改的单元格信息记录到modifiedCells列表中
private void gridView1_CellValueChanged(object sender,
DevExpress.XtraGrid.Views.Base.CellValueChangedEventArgs e)
{

    if (gridView1 != null)
    {
        // 获取发生变化的单元格所在的行和列的索引
        int rowIndex = e.RowHandle;
        int colIndex = e.Column.AbsoluteIndex;
        // 获取新值
        string newValue = e.Value.ToString();
        // 将信息存储到 pRowAndCol 数组中
        pRowAndCol[count].Row = rowIndex;
        pRowAndCol[count].Column = colIndex;
    }
}

```

```

        pRowAndCol[count].Value = newValue;
        // 增加 count 的值，为下一次存储做准备
        count++;
    }
}

```

// 保存编辑

```

private void SaveEdit()
{
    int selectIndex = cbxLayerSelect.SelectedIndex;
    ILayer pLayer = m_hookHelper.FocusMap.get_Layer(selectIndex);
    IFeatureLayer pFLayer = pLayer as IFeatureLayer;
    IFeatureClass pFeatureClass = pFLayer.FeatureClass;
    ITable pTable;
    //pTable = pFeatureClass.CreateFeature().Table;//很重要的一种获取shp表
    格的一种方式

    pTable = pFLayer as ITable;
    //将改变的记录值传给shp中的表
    int i = 0;
    while (pRowAndCol[i].Column != 0 || pRowAndCol[i].Row != 0)
    {
        IRow pRow;
        pRow = pTable.GetRow(pRowAndCol[i].Row+1);
        pRow.set_Value(pRowAndCol[i].Column, pRowAndCol[i].Value);
        pRow.Store();
        i++;
    }
    count = 0;
    for (int j = 0; j < i; j++)
    {
        pRowAndCol[j].Row = 0;
        pRowAndCol[j].Column = 0;
        pRowAndCol[j].Value = null;
    }

    m_mapControl.Refresh();
}

```

// 删除要素按钮点击事件处理函数，获取表格中选中的行，将对应行索引添加到要删除的行列表中，并从表格中删除对应行

```
private void btnDelFeature_ItemClick(object sender, ItemClickEventArgs e)
```

```

{
    GridView gridView = gridControl1.FocusedView as GridView;
    if (gridView != null)
    {
        int[] selectedRows = gridView.GetSelectedRows();
        if (selectedRows.Length > 0)
        {
            for (int i = selectedRows.Length - 1; i >= 0; i--)
            {
                int rowHandle = selectedRows[i];
                var row = gridView.GetRow(rowHandle);

                if (row != null)
                {
                    string strInt=gridView.GetRowCellValue(rowHandle,
gridView.Columns[0]).ToString();
                    int firstColumnValue = Convert.ToInt32(strInt);
                    rowsToDelete.Add(firstColumnValue);
                }
                gridView.DeleteRow(rowHandle);
            }
        }
    }
}

```

```

private void SaveDelete()
{
    int selectedIndex = cbxLayerSelect.SelectedIndex;
    ILayer pLayer = m_hookHelper.FocusMap.get_Layer(selectedIndex);
    IFeatureLayer pFLayer = pLayer as IFeatureLayer;
    IQueryFilter queryFilter = new QueryFilterClass();
    queryFilter.WhereClause = "";
    IFeatureClass pFeatureClass = pFLayer.FeatureClass;
    ITable pTable;
    pTable = pFLayer as ITable;
    if (rowsToDelete.Count > 0)
    {
        IWorkspaceEdit workspaceEdit = (pTable as IDataset).Workspace as
IWorkspaceEdit;
        // try
        // {
            workspaceEdit.StartEditing(true);
            workspaceEdit.StartEditOperation();
        }
    }
}

```

```
// 保存删除的要素

        foreach (int rowHandle in rowsToDelete)
        {
            IRow pRow = pTable.GetRow(rowHandle);
            if (pRow != null)
            {
                pRow.Delete();
            }
        }

        workspaceEdit.StopEditOperation();
        workspaceEdit.StopEditing(true);
        rowsToDelete.Clear();

        // }
        // catch (Exception ex)
        // {
        //     workspaceEdit.StopEditOperation();
        //     workspaceEdit.StopEditing(false);
        //     MessageBox.Show($"删除操作出现错误: {ex.Message}", "错误",
        //     MessageBoxButtons.OK, MessageBoxIcon.Error);
        // }
    }
}
```

4. 2. 1. 3 属性表选择高亮

```
private void HighlightFeaturesByQuery(IQueryFilter queryFilter)
{
    ILayer mLayer;
    // 获取当前在图层选择下拉框中选中的图层
    mLayer = m_hookHelper.FocusMap.get_Layer(cbxLayerSelect.SelectedIndex);
    IFeatureLayer pFLayer = mLayer as IFeatureLayer;
    if (pFLayer != null)
    {
        IFeatureSelection pFeatSelection;
        // 将要素图层转换为要素选择对象，用于后续根据条件选择要素
        pFeatSelection = pFLayer as IFeatureSelection;
        // 根据传入的查询条件选择要素，这里选择的是新建选择集（替换之前的选择结果），并且不进行扩展选择
        pFeatSelection.SelectFeatures(queryFilter,
        esriSelectionResultEnum.esriSelectionResultNew, false);

        IViewRefresh viewRefresh = m_mapControl.Map as IViewRefresh;
        // 设置渐进式绘制为真，可能用于优化地图刷新显示效果，尤其是在处理大量要素时
        viewRefresh.ProgressiveDrawing = true;
        // 刷新指定的图层，使得地图上对应的要素高亮显示效果生效
    }
}
```

```

        viewRefresh.RefreshItem(mLayer);
    }
}

// 表格行点击事件处理函数，根据点击行获取相关条件，构建查询条件并在地图上高亮显示对应要素，或者清除高亮显示（如果没有有效条件）
private void GridViewRowClickHandler(object sender, RowClickEventArgs e)
{
    GridView gridView = sender as GridView;
    if (gridView != null)
    {
        IQueryFilter pQuery = new QueryFilterClass();
        int count = gridView.SelectedRowCount;

        // 获取第一列的字段名
        string firstColumnName = gridView.Columns[0].FieldName;
        string[] possibleColumns = { firstColumnName };
        string val = "";

        // 遍历可能用于构建查询条件的列（这里是OBJECTID相关列），尝试获取对应单元格的值作为查询条件
        foreach (string col in possibleColumns)
        {
            try
            {
                if (count == 1)
                {
                    val =
gridView.GetRowCellValue(gridView.GetSelectedRows()[0], col).ToString();
                    if (!string.IsNullOrEmpty(val))
                    {
                        break;
                    }
                }
                else
                {
                    for (int i = 0; i < count; i++)
                    {
                        string tempVal =
gridView.GetRowCellValue(gridView.GetSelectedRows()[i], col).ToString();
                        if (!string.IsNullOrEmpty(tempVal))
                        {
                            val = tempVal;
                            break;
                        }
                    }
                }
            }
            catch { }
        }
    }
}

```

```

        }
    }
    if (!string.IsNullOrEmpty(val))
    {
        break;
    }
}
}
catch
{
    continue;
}
}

if (!string.IsNullOrEmpty(val))
{
    if (count == 1)
    {
        // 如果只选中了一行，构建简单的相等查询条件（例如 OBJECTID = 具体值）

        pQuery.WhereClause = possibleColumns[0] + "=" + val;
        HighlightFeaturesByQuery(pQuery);
    }
    else
    {
        string condition = "";
        // 如果选中了多行，构建 OR 连接的多个相等查询条件（例如 OBJECTID = 值1 OR OBJECTID = 值2 等）
        foreach (int rowHandle in gridView.GetSelectedRows())
        {
            string rowVal = gridView.GetRowCellValue(rowHandle, possibleColumns[0]).ToString();
            condition += possibleColumns[0] + "=" + rowVal + " OR ";
        }
        if (condition.Length > 0)
        {
            // 移除最后多余的 " OR " 字符串
            condition = condition.Remove(condition.Length - 4);
        }
        pQuery.WhereClause = condition;
        HighlightFeaturesByQuery(pQuery);
    }
}
else

```

```

        {
            // 如果没有获取到有效的查询条件值，则清除地图上的高亮显示
            ClearHighlight();
        }
    }
}

```

4.2.2 公交站点查询：

4.2.2.1 缩放到要素位置

```

private void ZoomToGeometry(ESRI.ArcGIS.Geometry.IGeometry geometry, IActiveView
activeView)
{
    IEnvelope env = geometry.Envelope;
    IGeometry5 geo = geometry as IGeometry5;

    if (env.Width == 0 || env.Height == 0)
    {
        ESRI.ArcGIS.Geometry.IPoint centerPoint = new PointClass();
        centerPoint.X = geo.CentroidEx.X;
        centerPoint.Y = geo.CentroidEx.Y;

        //IDisplayTransformation displayTrans =
activeView.ScreenDisplay.DisplayTransformation;
        //displayTrans.SpatialReference = activeView.FocusMap.SpatialReference;

        env.XMin = centerPoint.X - 0.001;
        env.XMax = centerPoint.X + 0.001;
        env.YMin = centerPoint.Y - 0.001;
        env.YMax = centerPoint.Y + 0.001;

        env.CenterAt(centerPoint);
    }
    env.Expand(3, 3, true);
    activeView.Extent = env;
    activeView.Refresh();
    activeView.ScreenDisplay.UpdateWindow();
}

```

4.2.2.2 空间查询

```

private void btnSearch_Click(object sender, EventArgs e)
{
    // “确认”按钮实现的功能【查询地点位置并缩放到该地点】
    string text = searchBusStops.Text.Trim();
}

```

```
IFeatureLayer pFeatureLayer = busStopLayer;

IQueryFilter queryFilter = new QueryFilterClass();

queryFilter.WhereClause = "name_st='" + text + "'";

IActiveView activeView = m_hookHelper.ActiveView;

try
{
    IFeatureCursor featureCursor = pFeatureLayer.Search(queryFilter,
false);

    IFeature pFeature;
    while ((pFeature = featureCursor.NextFeature()) != null)
    {
        ZoomToGeometry(pFeature.Shape, activeView);
        axMapControl.FlashShape(pFeature.Shape);
    }
}
catch (Exception pException)
{
    MessageBox.Show(pException.Message);
}

}

private void searchMetro_EditValueChanged(object sender, EventArgs e)
{
    try
    {
        if (searchMetroStops.EditValue != null)
        {
            string searchText = searchMetroStops.EditValue.ToString();
            List<string> filteredNames = new List<string>();
            if (string.IsNullOrEmpty(searchText))
            {
                LoadMetroStopData();
            }
            else
            {
                // 过滤数据
                List<string> allNames =
(List<string>)listMetroStops.DataSource;
```

```
        foreach (string name in allNames)
        {
            if (name.Contains(searchText))
            {
                filteredNames.Add(name);
            }
        }
        listMetroStops.DataSource = filteredNames;
    }
}

catch
{
}
}
```

4.2.3 噪音分析

4.2.3.1 线密度计算

```
private void TiffPush()
{
    string path = txtPushPath.Text;
    path = path.Replace("/", "\\");
    IMap map = m_hookHelper.FocusMap;
    IActiveView activeView = map as IActiveView;
    IEnvelope extent = activeView.Extent;
    ILayer selectedLayer = map.get_Layer(cbxLayerSelect.SelectedIndex);
    IFeatureLayer selectedFeatureLayer = selectedLayer as IFeatureLayer;
    IFeatureClass selectedFeatureClass = selectedFeatureLayer.FeatureClass;

    IRasterAnalysisEnvironment rasterEnv = new RasterDensityOp();

    //设置半径
    double r = Convert.ToDouble(txtRadius.Text);
    rasterEnv.SetCellSize(esriRasterEnvSettingEnum.esriRasterEnvValue, r);

    //设置输出栅格大小
    double cellSize = Convert.ToDouble(txtPixel.Text);
    object cellSizeObj = cellSize;
    rasterEnv.SetCellSize(esriRasterEnvSettingEnum.esriRasterEnvValue, ref
cellSizeObj);
}
```

```
//获得图层
ILayer layer = null;
IMap mapTemp = map;
for (int i = 0; i < mapTemp.LayerCount; i++)
{
    ILayer temp = mapTemp.Layer[i];
    if (temp.Name == cbxLayerSelect.SelectedItem.ToString())
        layer = temp;
}
IFeatureLayer fLayer = layer as IFeatureLayer;
IFeatureClass fClass = fLayer.FeatureClass;

//设置空间处理范围
IEnvelope fullExtent = m_hookHelper.ActiveView.FullExtent;
object extentProObj = fullExtent;
rasterEnv.SetExtent(esriRasterEnvSettingEnum.esriRasterEnvValue, ref
extentProObj);

//设置要素数据
IFeatureClassDescriptor feaDes;
feaDes = new FeatureClassDescriptorClass();
feaDes.Create(fClass, null, cbxFieldSelect.SelectedItem.ToString());
IGeoDataset inGeodataset;
inGeodataset = feaDes as IGeoDataset;

//设置输出栅格
IRaster outraster;
IGeoDataset outGeoDataset;

IDensityOp densityOp = rasterEnv as IDensityOp;
outGeoDataset = densityOp.KernelDensity(inGeodataset, r);
outraster = (IRaster)outGeoDataset;
try
{
    IRaster2 outputRaster = outraster as IRaster2;
    IRasterDataset rasterDataset = outputRaster.RasterDataset;
    IWorkspaceFactory pWKSF = new RasterWorkspaceFactoryClass();
    //path = path + "\\\"+ txtTiffName1.Text;
    IWorkspace pWorkspace = pWKSF.OpenFromFile(path, 0);
    ISaveAs2 pSaveAs = rasterDataset as ISaveAs2;
    string name = txtTiffName1.Text + ".tif";
    string fullPath = System.IO.Path.Combine(path, name);
    if (System.IO.File.Exists(fullPath))
    {
```

```

        System.IO.File.Delete(fullPath);
    }
    pSaveAs.SaveAs(name, pWorkspace, "TIFF");
    IRasterLayer pRlayer = new RasterLayer();
    pRlayer.CreateFromRaster(outputRaster as IRaster);
    pRlayer.Name = txtTiffName1.Text;
    map.AddLayer(pRlayer);
    cbxTiffSelect1.Properties.Items.Add(pRlayer.Name);
    cbxTiffSelect2.Properties.Items.Add(pRlayer.Name);
}

catch (Exception ex)
{
    MessageBox.Show(ex.ToString());
}

}

```

4. 2. 3. 2 栅格叠加

```

private void RasterAnalyse()
{
    string pythonPath=null;
    string scriptPath=null;
    PathPython(ref pythonPath, ref scriptPath);

    Process process = new Process();
    process.StartInfo.FileName = pythonPath;
    process.StartInfo.Arguments = scriptPath;
    process.StartInfo.UseShellExecute = false;
    process.StartInfo.RedirectStandardOutput = true;
    process.StartInfo.RedirectStandardError = true;
    process.StartInfo.CreateNoWindow = true;
    string arguments = $"{scriptPath} \"{rasterPath1}\" \"{rasterPath2}\"
    \"{raster1Coefficient}\" \"{raster2Coefficient}\" \"{rasterName}\" \"{rasterPath}\"";

    process.StartInfo.Arguments = arguments;
    try
    {
        process.Start();

        string output = process.StandardOutput.ReadToEnd();
        string error = process.StandardError.ReadToEnd();

        process.WaitForExit();
    }
}

```

```
        if (!string.IsNullOrEmpty(output))
        {
            MessageBox.Show("Python 输出: " + output);
        }
        if (!string.IsNullOrEmpty(error))
        {
            MessageBox.Show("Python 输出: " + error);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("调用 Python 时出错: " + ex.Message);
    }
}
```

```
# -*- coding: utf-8 -*-
import arcpy
from arcpy.sa import *
import sys
# 检查并启用 Spatial Analyst 扩展
arcpy.CheckOutExtension("Spatial")
arcpy.env.overwriteOutput = True
# 设置工作空间
arcpy.env.workspace = sys.argv[6].replace("\\", "/")

# 输入栅格数据的路径
input_raster1 = sys.argv[1].replace("\\", "/")
input_raster2 = sys.argv[2].replace("\\", "/")
# 为输入栅格 1 创建影像金字塔
arcpy.BuildPyramids_management(input_raster1)
# 为输入栅格 2 创建影像金字塔
arcpy.BuildPyramids_management(input_raster2)

# 定义权重
weight1 = float(sys.argv[3])
weight2 = float(sys.argv[4])

# 执行栅格加权和计算
output_raster = (arcpy.Raster(input_raster1) * weight1) + (arcpy.Raster(input_raster2)
* weight2)

# 保存结果栅格
```

```
output_raster.save(sys.argv[5]+".tif")
```

```
# 释放 Spatial Analyst 扩展
arcpy.CheckInExtension("Spatial")
```

4. 2. 4 栅格渲染

4. 2. 4. 1 拉伸渲染

```
private void ChangeRender2RasterStretchColorRampRender(IRasterLayer rasterLayer)
{
    if (rasterlayer == null)
        return;

    //计算统计图
    IRasterBand band = GetBand(rasterLayer);
    if (band.Histogram == null)
    {
        band.ComputeStatsAndHist();
    }

    IRaster raster = rasterLayer.Raster;
    IRasterStretchColorRampRenderer rasterStretchColorRampRenderer = new
RasterStretchColorRampRendererClass();
    IRasterRenderer rasterRenderer = rasterStretchColorRampRenderer as
IRasterRenderer;
    rasterRenderer.Raster = raster;
    rasterRenderer.Update();

    //IAlgorithmicColorRamp algorithmicColorRamp = new
AlgorithmicColorRampClass();
    //algorithmicColorRamp.Size = 255;
    //algorithmicColorRamp.FromColor = getRGBColor(255, 0, 0);
    //algorithmicColorRamp.ToColor = getRGBColor(0, 255, 0);
    //bool btrue;
    //algorithmicColorRamp.CreateRamp(out btrue);

    rasterStretchColorRampRenderer.BandIndex = cmbRasterBands.SelectedIndex;
    IColorRamp pColorRamp =
(IColorRamp)EnumStyleItem[colorComboBox.SelectedIndex];
    rasterStretchColorRampRenderer.ColorRamp = pColorRamp;
    rasterRenderer.Update();
    rasterLayer.Renderer = rasterStretchColorRampRenderer as IRasterRenderer;

    axmapcontrol.ActiveView.PartialRefresh(esriViewDrawPhase.esriViewGeography,
```

```

null, null);
        axtocontrol.Update();
        axmapcontrol.Extent = rasterlayer.AreaOfInterest;
    }

```

4.2.4.2 分级渲染

```

public void RasterClassify(IRasterLayer rastlayer, string classMethod, int count,
IColorRamp ramp)
{
    try
    {
        //计算统计图
        IRasterBand band = GetBand(rastlayer);
        if (band.Histogram == null)
        {
            band.ComputeStatsAndHist();
        }
        IRasterClassifyColorRampRenderer rasClassifyRender = new
RasterClassifyColorRampRendererClass();
        IRasterRenderer rasRender = rasClassifyRender as IRasterRenderer;
        rasRender.Raster = rastlayer.Raster;
        rasRender.Update();

        int numClasses = count;
        IRasterHistogram pRasterHistogram = band.Histogram;
        double[] dblValues = pRasterHistogram.Counts as double[];
        int intValueCount = dblValues.GetUpperBound(0) + 1;
        double[] vValues = new double[intValueCount];
        IRasterStatistics pRasterStatistic = band.Statistics;
        double dMaxValue = pRasterStatistic.Maximum;
        double dMinValue = pRasterStatistic.Minimum;

        double BinInterval = Convert.ToDouble((dMaxValue - dMinValue) /
intValueCount);
        for (int i = 0; i < intValueCount; i++)
        {
            vValues[i] = i * BinInterval + pRasterStatistic.Minimum;
        }
        long[] longvalues = new long[dblValues.Length];
        for (int i = 0; i <= dblValues.Length - 1; i++)
        {
            longvalues[i] = long.Parse(Convert.ToString(dblValues[i]));
        }
        //IClassifyGEN classify = null;
        IClassify classify = null;
    }
}

```

```

switch (classMethod)
{
    case "等间距分级":
        classify = new EqualIntervalClass();
        break;
    case "自然断点分级":
        classify = new NaturalBreaksClass();
        break;
}
classify.Classify(ref numClasses);

double[] Classes = classify.ClassBreaks as double[];
UID pUid = classify.ClassID;
IRasterClassifyUIProperties rasClassifyUI = rasClassifyRender as
IRasterClassifyUIProperties;
rasClassifyUI.ClassificationMethod = pUid;
rasClassifyRender.ClassCount = count;
IColor pColor;
ISimpleFillSymbol pSym;

for (int j = 0; j < count; j++)
{
    pColor = ramp.get_Color(j * (ramp.Size - 1) / (count - 1));
    pSym = new SimpleFillSymbolClass();
    pSym.Color = pColor;
    rasClassifyRender.set_Symbol(j, (ISymbol)pSym);
    rasRender.Update();
    rasClassifyRender.set_Break(j, rasClassifyRender.get_Break(j));
    //rasClassifyRender.set_Label(j, Classes[j].ToString("0.000") + "-"
+ Classes[j + 1].ToString("0.000"));
    rasRender.Update();
}

//IRasterProps rasterProps = (IRasterProps)rastlayer.Raster;
//rasterProps.NoDataValue = 0;
//IRasterDisplayProps props = rasClassifyRender as IRasterDisplayProps;
//props.NoDataColor = GET(255, 255, 255);
//rasRender.Update();
rastlayer.Renderer = rasClassifyRender as IRasterRenderer;
}
catch
{
    MessageBox.Show("唯一值数量已达到限制 (65536)");
}

```

```

    }
}

```

4.2.5 最优路径分析

4.2.5.1 打开网络数据集并添加路径

```

private void LoadNetwork()
{
    folderBrowserDialog1.SelectedPath = Application.StartupPath; ;
    if (folderBrowserDialog1.ShowDialog() != DialogResult.OK) return;

    string folderPath = folderBrowserDialog1.SelectedPath;

    //打开工作区间
    NetworkAnalysis.pFWorkspace = NetworkAnalysis.OpenWorkspace(folderPath) as
IFeatureWorkspace;
    //打开网络数据集
    NetworkAnalysis.networkDataset =
NetworkAnalysis.OpenNetworkDataset(NetworkAnalysis.pFWorkspace as IWorkspace, "路网
_ND", "路网");
    //NetworkAnalysis.networkDataset =
NetworkAnalysis.OpenNetworkDataset(NetworkAnalysis.pFWorkspace as IWorkspace, "同济新村
道路_ND", "同济新村道路");
    //创建网络分析上下文，建立一种解决关系
    NetworkAnalysis.m_NAContext =
NetworkAnalysis.CreateSolverContext(NetworkAnalysis.networkDataset);

    //打开停靠点数据集
    NetworkAnalysis.inputFCClass =
NetworkAnalysis.pFWorkspace.OpenFeatureClass("stops");

    //TEST_ND_JUNCTIONS图层
    IFeatureLayer vertex = new FeatureLayerClass();
    vertex.FeatureClass = NetworkAnalysis.pFWorkspace.OpenFeatureClass("路网
_ND_Junctions");
    //vertex.FeatureClass = NetworkAnalysis.pFWorkspace.OpenFeatureClass("同济新
村道路_ND_Junctions");
    vertex.Name = vertex.FeatureClass.AliasName;
    //axMapControl.AddLayer(vertex, 0);
    //道路图层
    IFeatureLayer road;
    road = new FeatureLayerClass();
    road.FeatureClass = NetworkAnalysis.pFWorkspace.OpenFeatureClass("路网");
    //road.FeatureClass = NetworkAnalysis.pFWorkspace.OpenFeatureClass("同济新
村道路");
}

```

```

road.Name = road.FeatureClass.AliasName;
//axMapControl.AddLayer(road, 0);

//为networkdataset生成一个图层,并将该图层添加到axmapcontrol中
ILayer pLayer;//网络图层
INetworkLayer pNetworkLayer;
pNetworkLayer = new NetworkLayerClass();
pNetworkLayer.NetworkDataset = NetworkAnalysis.networkDataset;
pLayer = pNetworkLayer as ILayer;
pLayer.Name = "成都市路网";
axMapControl.AddLayer(pLayer, 0);
//生成一个网络分析图层并添加到axmaptroll中
ILayer layer1;
INALayer nalayer =
NetworkAnalysis.m_NAContext.Solver.CreateLayer(NetworkAnalysis.m_NAContext);

layer1 = nalayer as ILayer;
layer1.Name = NetworkAnalysis.m_NAContext.Solver.DisplayName;
axMapControl.AddLayer(layer1, 0);
m_ipActiveView = axMapControl.ActiveView;
NetworkAnalysis.p2DMap = m_ipActiveView.FocusMap;
NetworkAnalysis.PGC = NetworkAnalysis.p2DMap as IGraphicsContainer;
}

```

4.2.5.2 路径分析

```

private void Analysis()
{
    if (NetworkAnalysis.networkAnalysis && NetworkAnalysis.clickedcount > 1)
    {
        NetworkAnalysis.networkAnalysis = false;
        listInfoNavigation.Items.Add("开始进行路径分析求解...");

        IGPMessages gpMessages = new GPMessagesClass();
        loadNANetworkLocations("stops", NetworkAnalysis.inputFCClass, 50000000);
        naSolver = NetworkAnalysis.m_NAContext.Solver;

        try
        {
            bool flag = naSolver.Solve(NetworkAnalysis.m_NAContext, gpMessages,
null);
        }
        catch
        {

```

```

        MessageBox.Show("未找到路径");
        return;
    }
    listInfoNavigation.Items.Add("done!");

    //获取路径结果
    INAStrreetDirectionsAgent2 streetAgent;
    streetAgent =
NetworkAnalysis.m_NAContext.Agents.get_ItemByName("StrreetDirectionsAgent") as
INAStrreetDirectionsAgent2;
    //设置处理结果为中文
    //streetAgent.Language = "zh-CN";

    streetAgent.Execute(null, null);

    INAStrreetDirectionsContainer directionsContainer;
    directionsContainer = streetAgent.DirectionsContainer as
INAStrreetDirectionsContainer;
    //directionsContainer.SaveAsXML("route.xml");

    int directionsCount = directionsContainer.DirectionsCount;
    string routeIdName = string.Empty;
    string routeString;
    for (int index = 0; index < directionsCount; index++)
    {
        INAStrreetDirections naStrreetDirections =
directionsContainer.get_Directions(index);
        routeString = getDirectionsString(naStrreetDirections);
        getDataTableFromRouteString(routeString);
    }

    //解决完后，删除图层内容
    ITable pTable_inputFCClass = NetworkAnalysis.inputFCClass as ITable;
    pTable_inputFCClass.DeleteSearchedRows(null);
    (NetworkAnalysis.p2DMap as IActiveView).Refresh();

    if (gpMessages != null)
    {
        for (int i = 0; i < gpMessages.Count; i++)
        {
            switch (gpMessages.GetMessage(i).Type)
            {
                case esriGPMessageType.esriGPMessageTypeError:

```

```

        listInfoNavigation.Items.Add("错误 " +
gpMessages.GetMessage(i).ErrorCode.ToString() + " " +
gpMessages.GetMessage(i).Description);
        break;
        case esriGPMessageType.esriGPMessageTypeWarning:
            listInfoNavigation.Items.Add("警告 " +
gpMessages.GetMessage(i).Description);
            break;
        default:
            listInfoNavigation.Items.Add("信息 " +
gpMessages.GetMessage(i).Description);
            break;
    }
}

(NetworkAnalysis.p2DMap as IActiveView).Refresh();

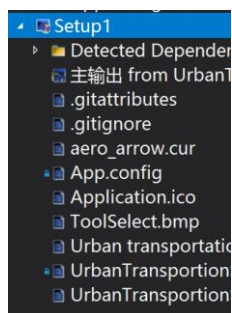
}
else if (NetworkAnalysis.clickedcount == 1)
{
    MessageBox.Show("未选取终点，请继续定位！");
}
else
{
    MessageBox.Show("未选取或更新起点和终点，请重新定位！");
}
}

```

5 系统运行

5.1 系统运行环境配置

5.1.1 Setup 配置



点击创建之后，生成安装包 setup1 点击运行可以将程序生成一个可执行的 exe 文件。

5.2 登录功能

5.2.1 界面演示

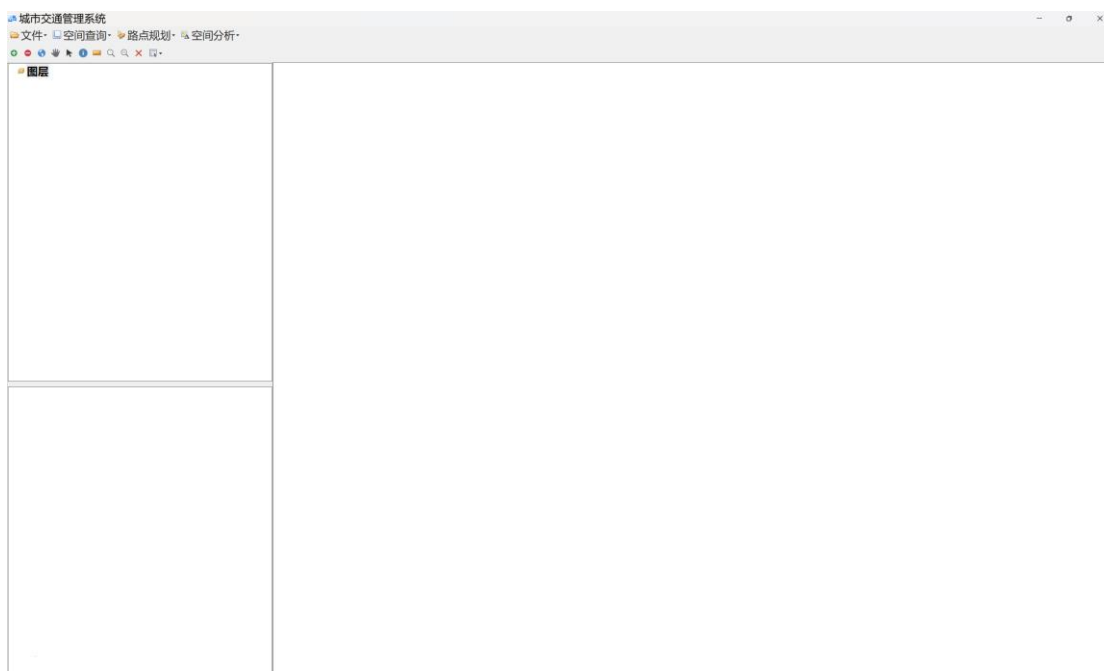


5.2.2 操作流程

用户输入账户信息→点击登录→进入系统

5.3 基础地理信息操作

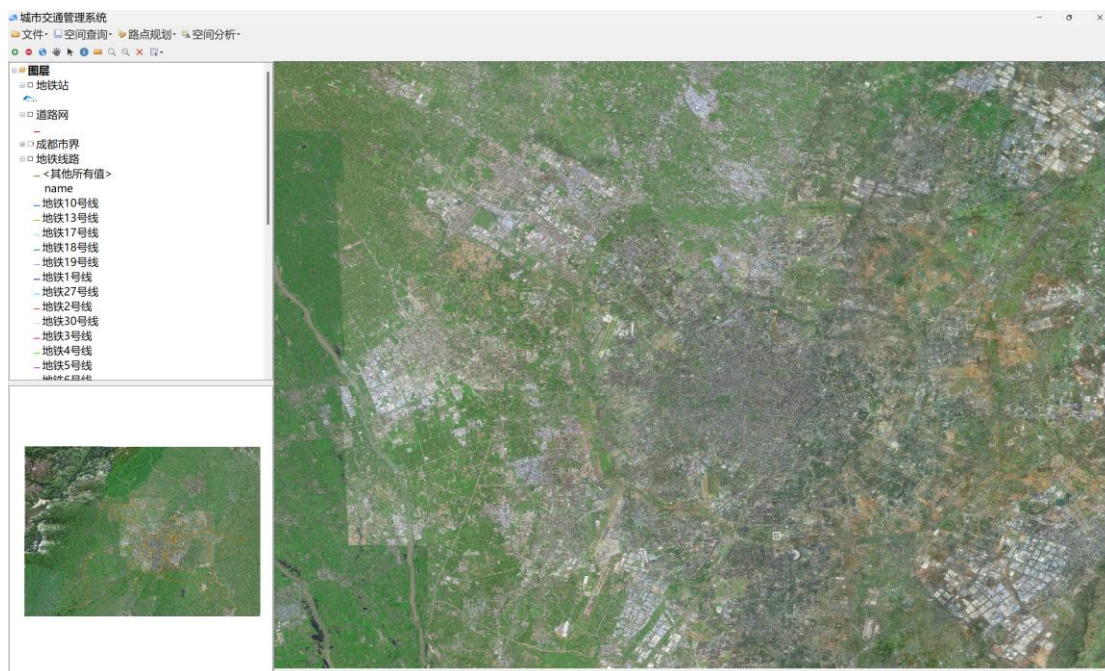
5.3.1 管理员系统界面



5.3.2 打开地图文档（mxd）

5.3.2.1 操作流程

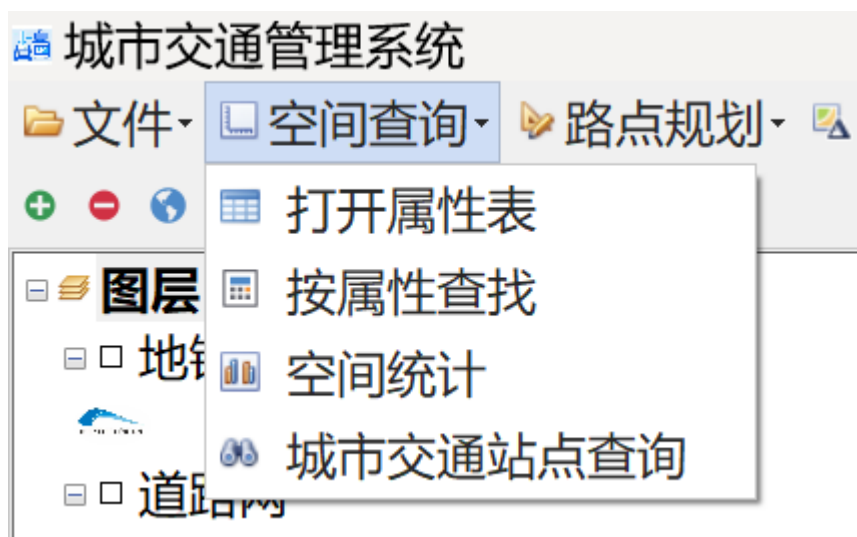
点击菜单栏“打开文档”按钮→选择文档→点击确定→地图文档添加



5.3.3 属性表浏览

5.3.3.1 方法1 操作流程

点击菜单栏“空间查询操作”→选择图层并显示



5.3.3.2 方法2 操作流程

右键菜单栏→打开属性表



5.3.3.3 效果演示

属性表

编辑状态 保存编辑 导出表格 删除要素 增加字段

选择图层: 地铁站

OBJECTID	Shape	name	Shape_Length
2	esriGeometryPolyline	地铁10号线	0.35525466672274
3	esriGeometryPolyline	地铁18号线	0.654463464618561
6	esriGeometryPolyline	地铁17号线	0.0579328543704113
8	esriGeometryPolyline	地铁27号线	0.236712324919403
9	esriGeometryPolyline	地铁1号线	0.341534675766972
11	esriGeometryPolyline	地铁6号线	0.656657628047653
13	esriGeometryPolyline	地铁5号线	0.446868522708787
16	esriGeometryPolyline	地铁9号线	0.20497457551132
17	esriGeometryPolyline	地铁2号线	0.410871568195052
19	esriGeometryPolyline	地铁8号线	0.283044756836308
21	esriGeometryPolyline	地铁30号线	0.275344657506346
23	esriGeometryPolyline	地铁7号线	0.38058515578177
26	esriGeometryPolyline	地铁4号线	0.430664522567123
28	esriGeometryPolyline	地铁13号线	0.280689506798371
30	esriGeometryPolyline	地铁19号线	0.95876198886355
32	esriGeometryPolyline	地铁7号线	0.380585171027241
33	esriGeometryPolyline	地铁3号线	0.477827320389053

5.4 空间查询

5.4.1 按属性查询

5.4.1.1 字段计算器界面

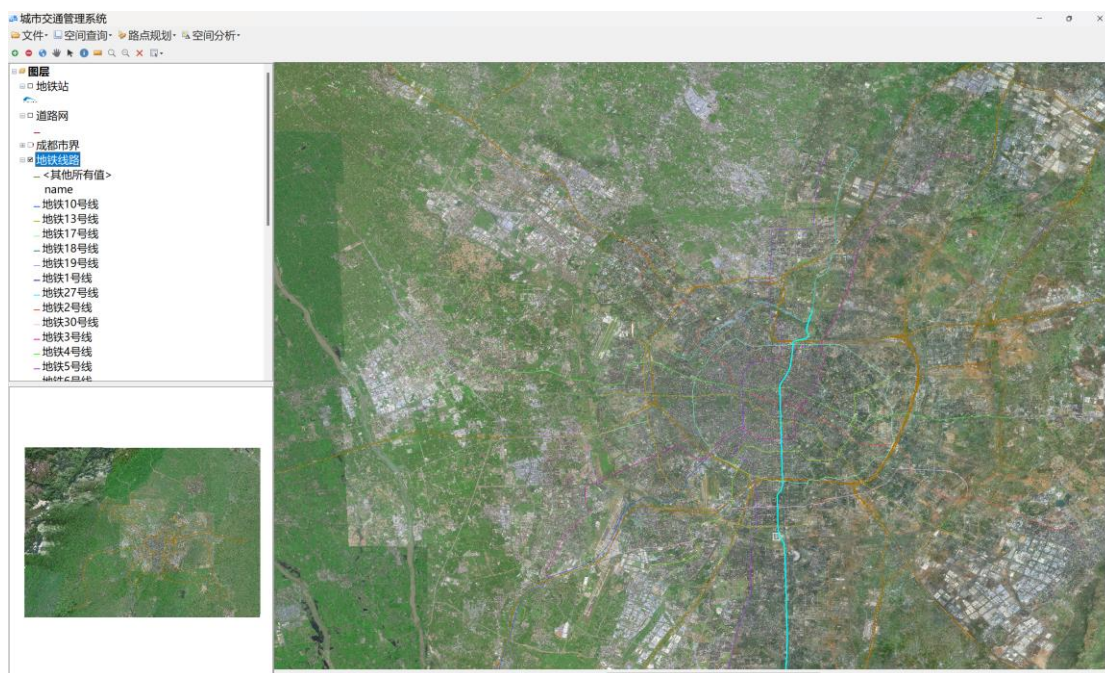
5.4.1.2 操作流程

选择图层→点击属性字段→双击属性字段→获取属性值→编写 sql 查询语句

5.4.1.3 运行结果

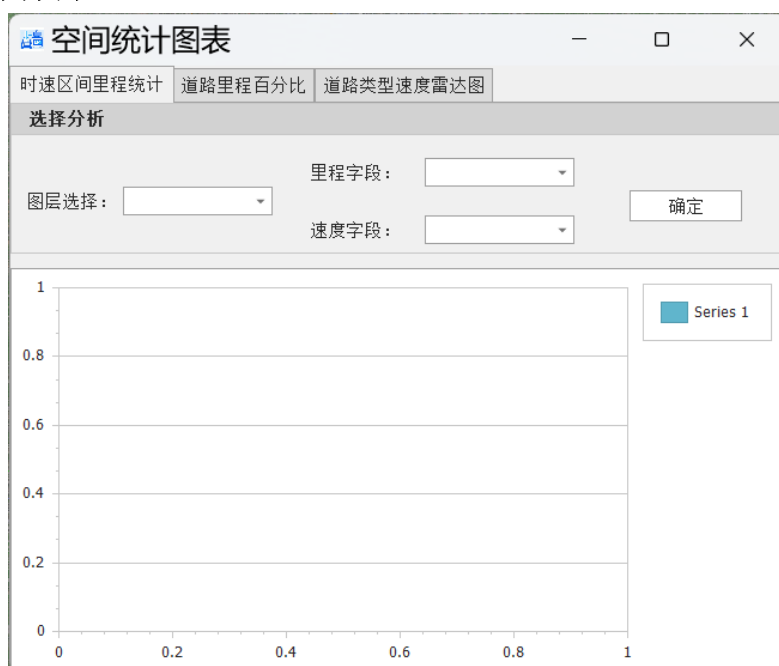
比如说要查找成都地铁 1 号线，可以在字段计算器中进行编辑，然后进行确定

地图高亮显示



5.4.2 空间统计

5.4.2.1 界面演示



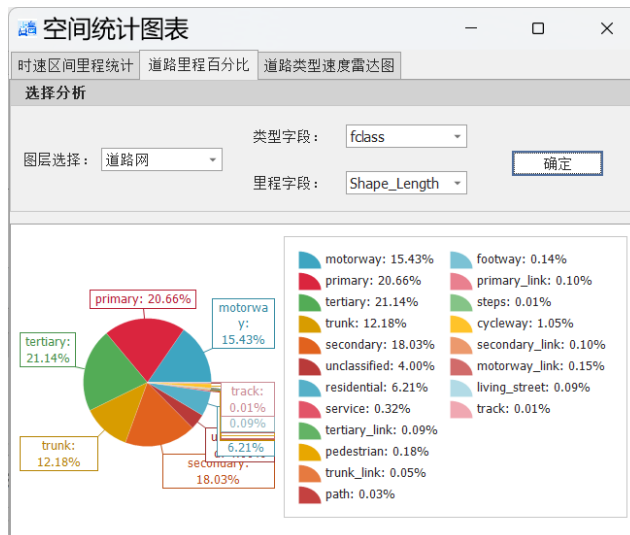
5.4.2.2 操作步骤

选择统计分析类型→选择图层→进行计算→分析图表

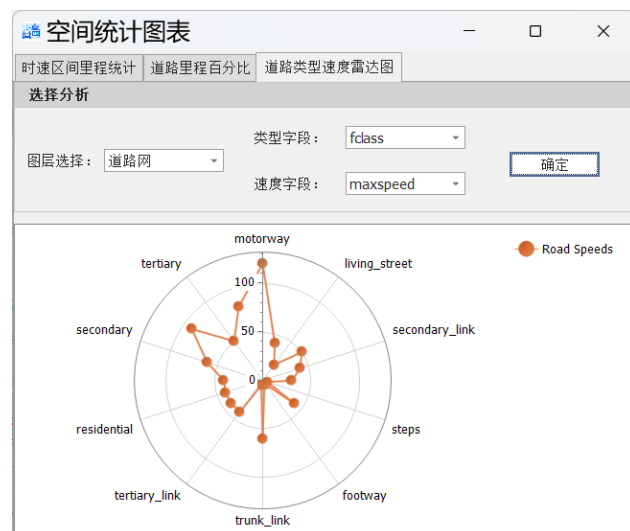
5. 4. 2. 3 时速区间里程统计



5. 4. 2. 4 道路里程百分比统计

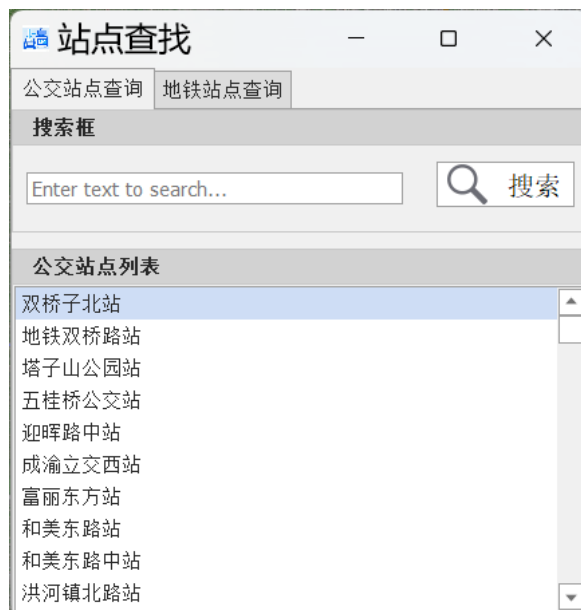


5. 4. 2. 5 道路类型速度雷达图统计



5.4.3 交通站点查询

5.4.3.1 界面演示

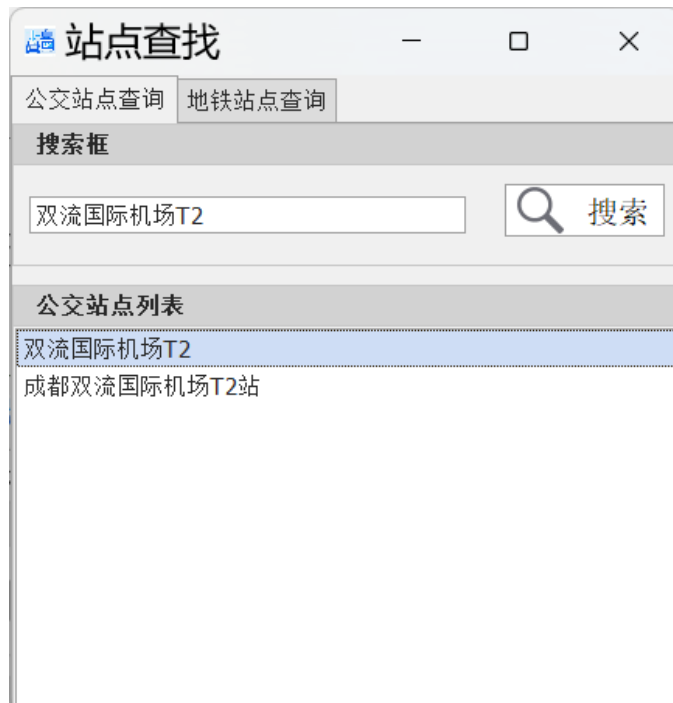


5.4.3.2 操作流程

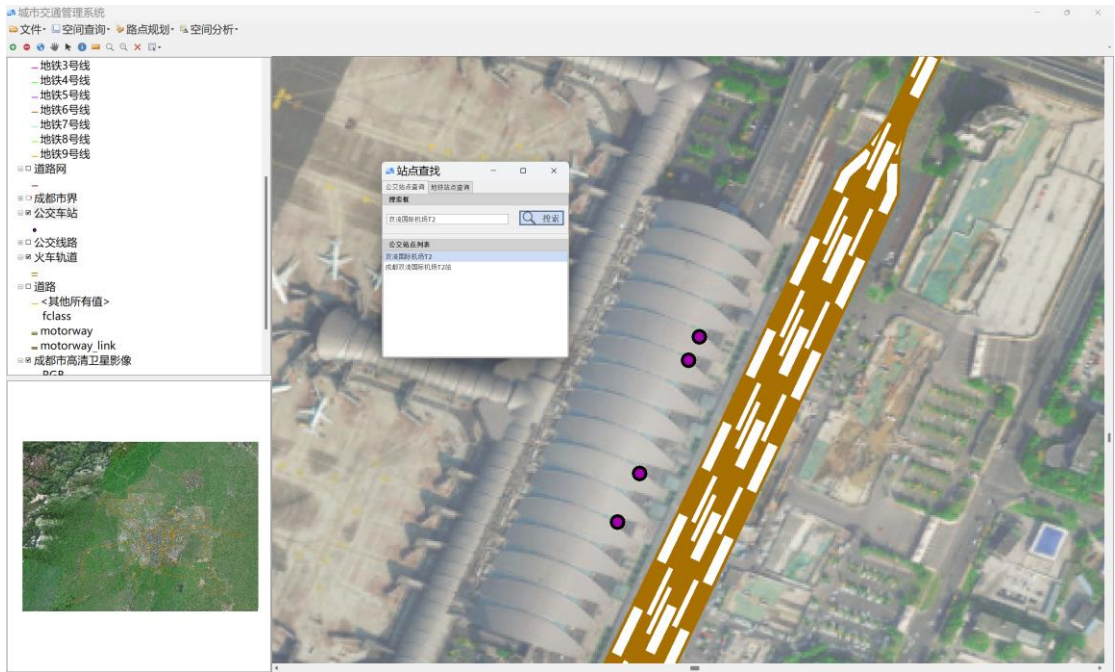
输入站名→搜索查询

5.4.3.3 运行演示

比如搜索双流机场的公交站



点击搜索后会缩放到要素位置并且要素会闪烁。



5.5 要素类创建与要素编辑

5.5.1 创建要素类

5.5.1.1 窗口界面



5.5.1.2 操作流程

选择要素类型→编辑字段（添加字段）→设置要素名→设置保存路径→创建图层

5.5.1.3 运行演示：

比如新建一个要素类为有轨电车站点：

创建新图层

类型

☐ 线路
 ☒ 站点

字段编辑器

字段名:

字段类型:

Text

添加字段

要素名

有轨电车站点

字段表

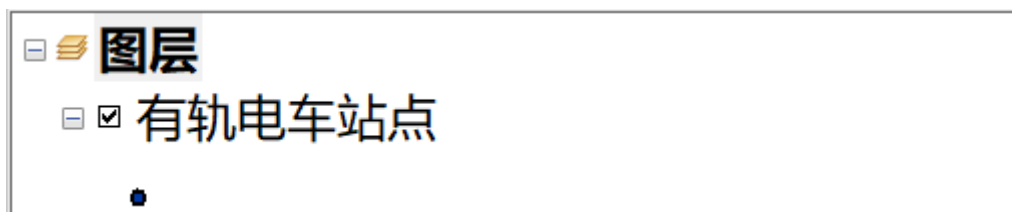
字段名称	字段类型
name	Text

保存路径

D:\arcmap\成都市.gdb

确定

图层栏:



属性表:

属性表

编辑状态

保存编辑

导出表格

删除要素

增加字段

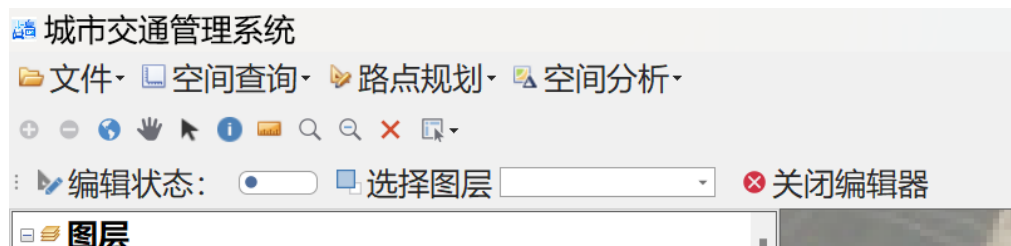
选择图层:

有轨电车站点

OBJECTID	Shape	name
----------	-------	------

5.5.2 矢量编辑器

5.5.2.1 工具栏展示

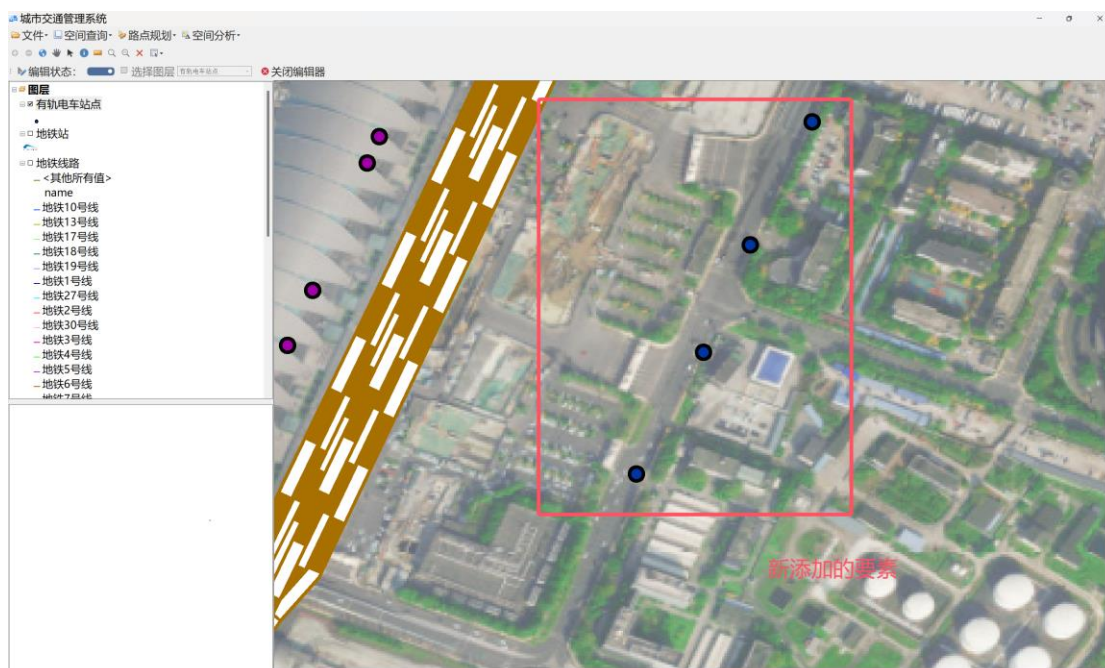


5.5.2.2 操作步骤

选择图层→打开编辑状态→要素编辑→关闭编辑状态→关闭编辑器

5.5.2.3 运行演示

对新建的有轨电车站进行编辑



5.5.3 属性表编辑

5.5.3.1 操作步骤

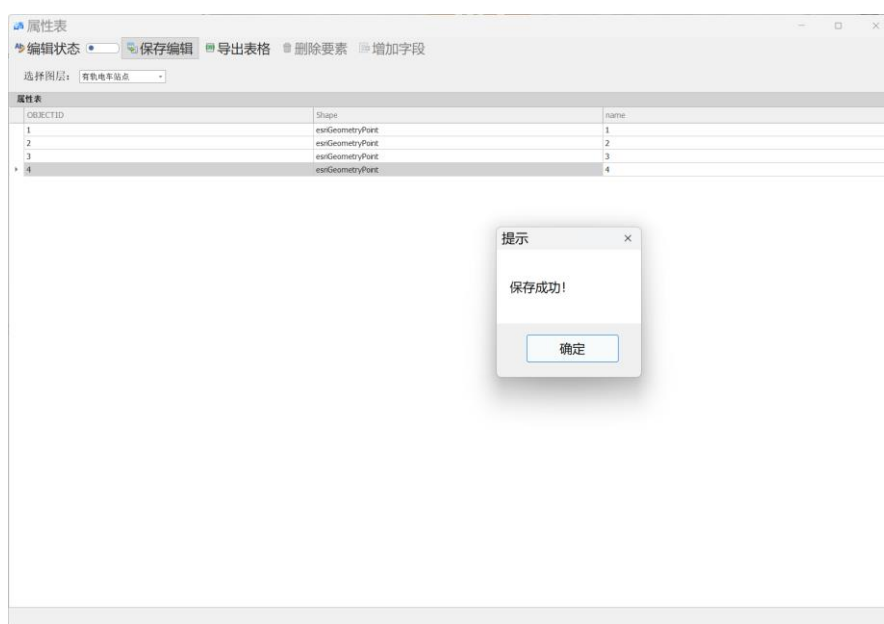
打开编辑状态→编辑→关闭编辑状态→保存编辑

5.5.3.2 效果演示

打开编辑状态：



编辑之后关掉编辑状态后点击保存



再次打开属性表，若成果保存，说明保存成功

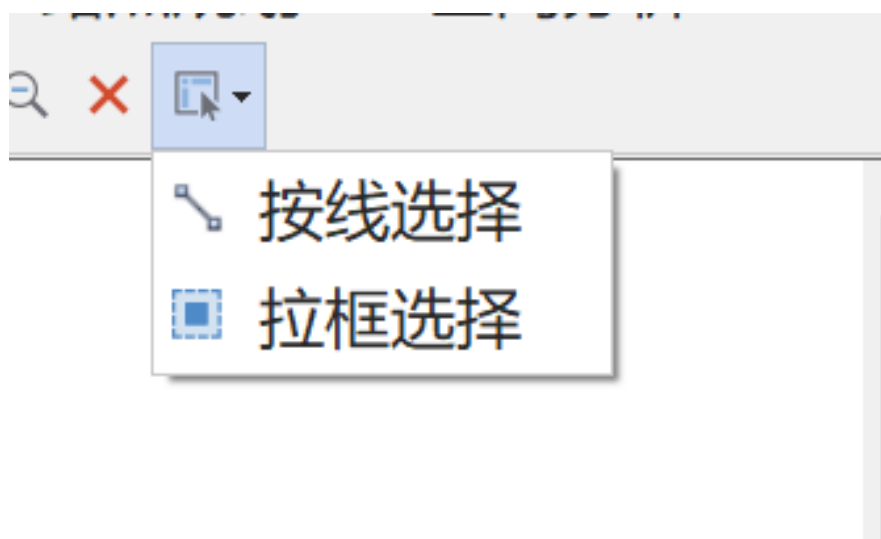
5.5.4 地图选择要素删除

5.5.4.1 操作步骤

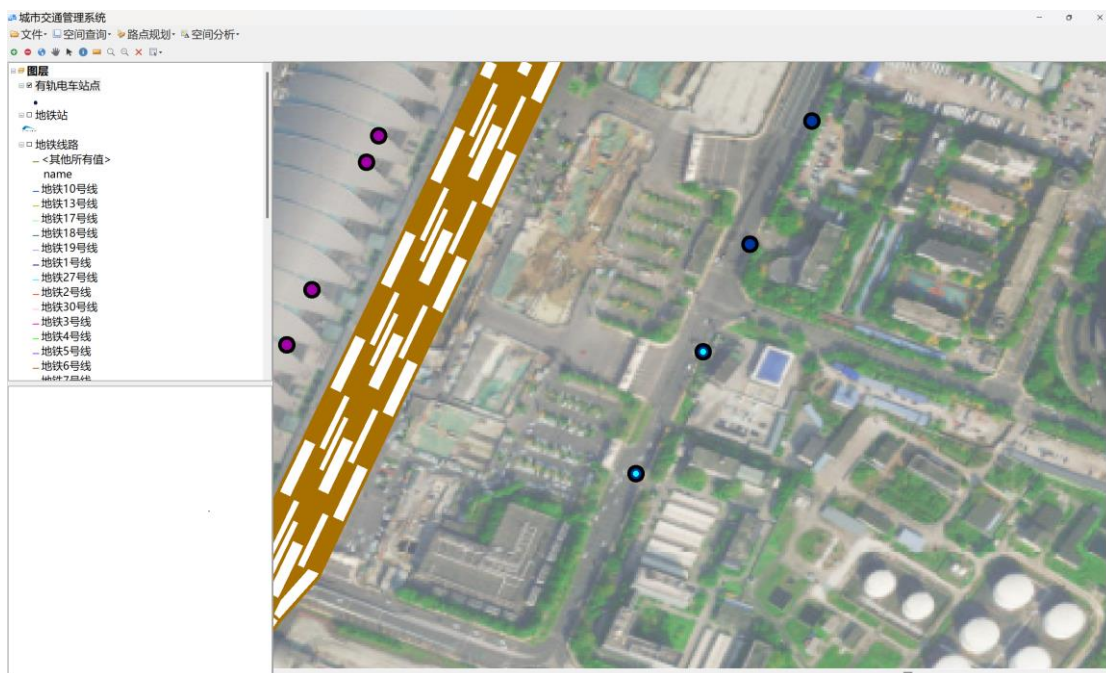
利用工具栏工具拉框选择，然后在点击删除

5.5.4.2 效果演示

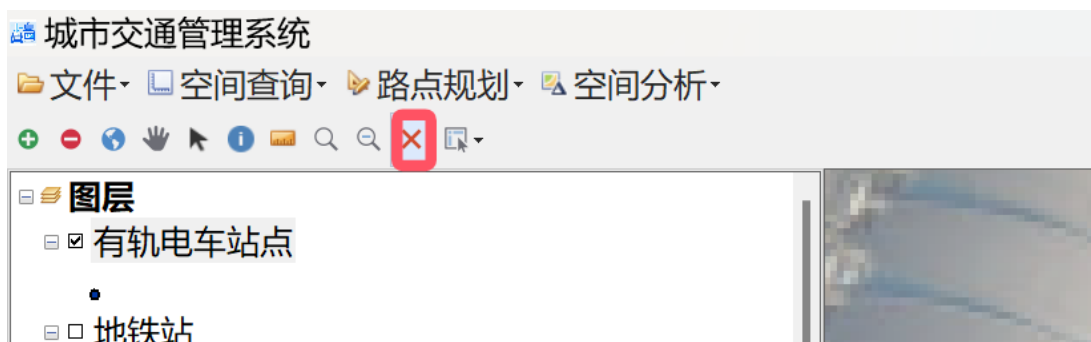
首先选择要素



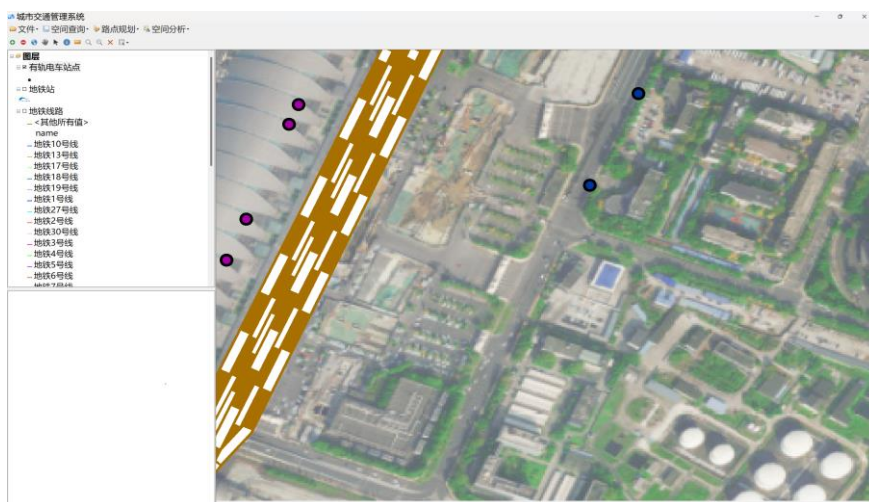
选择图上要删除的要素



点击删除



删除效果:



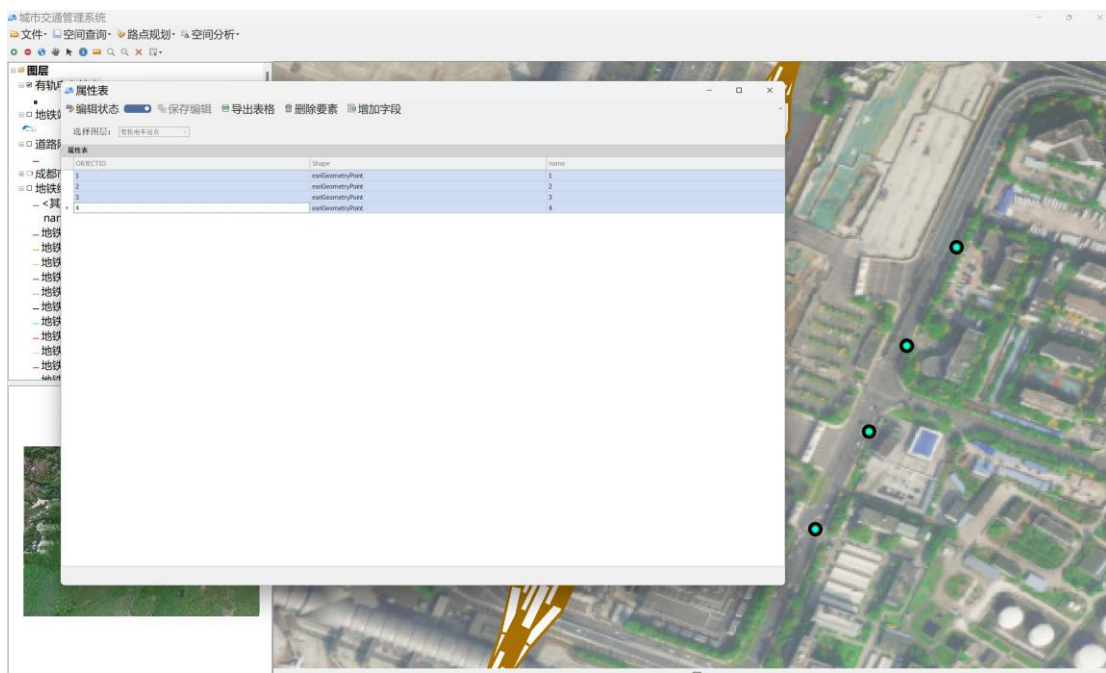
5.5.5 属性表批量删除要素

5.5.5.1 操作步骤

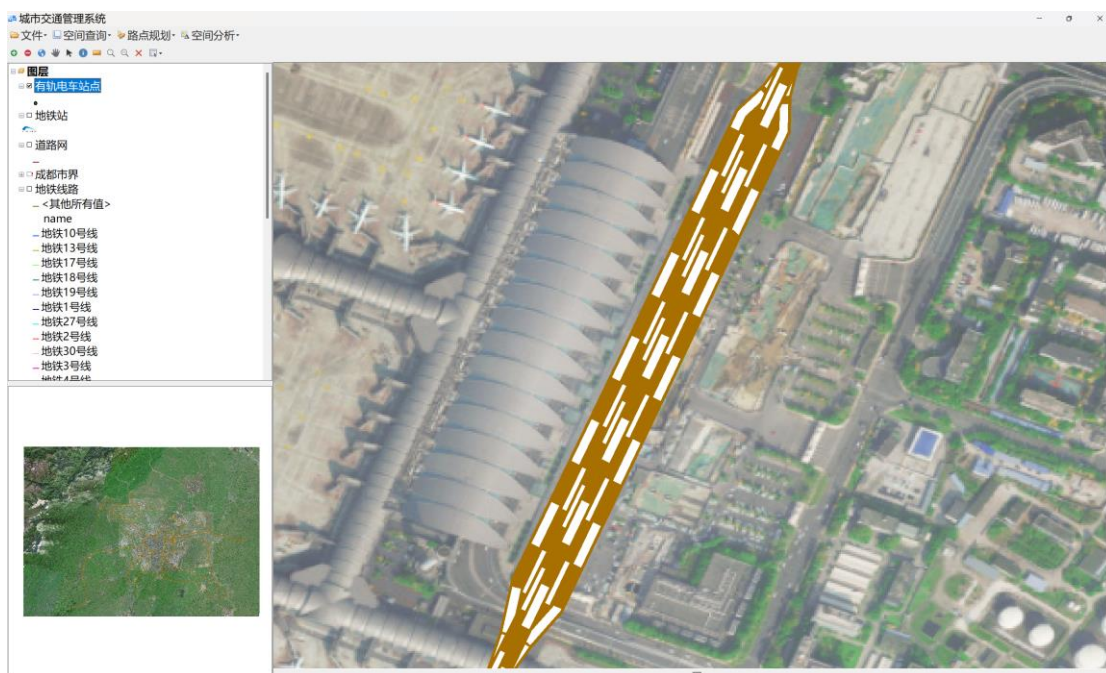
打开属性表→打开编辑状态→选择要素（行）→删除要素→保存编辑状态

5.5.5.2 运行演示

选择行：



删除效果：



5.6 空间分析

5.6.1 噪音计算（线密度分析）

5.6.1.1 界面

噪音分析

叠加分析 | 噪音计算

噪音计算

数据源设置：

道路图层：

噪音字段：

参数设置：

半径：

像素大小：

保存路径： ... 名称：

5.6.1.2 操作步骤

设置数据源→设置线密度分析参数→设置保存路径→设置名称

5.6.1.3 运行演示

设置

噪音分析

叠加分析 | 噪音计算

噪音计算

数据源设置：

道路图层：

噪音字段：

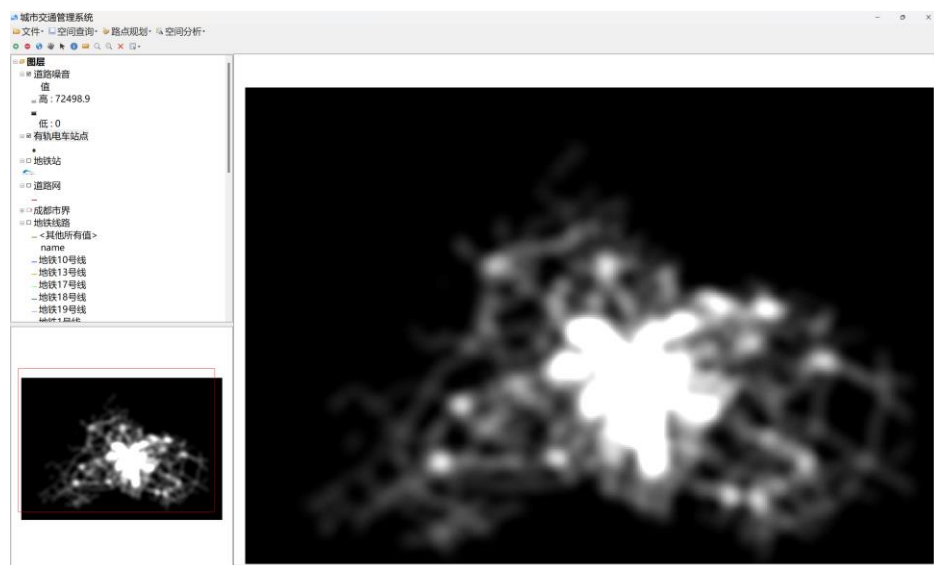
参数设置：

半径：

像素大小：

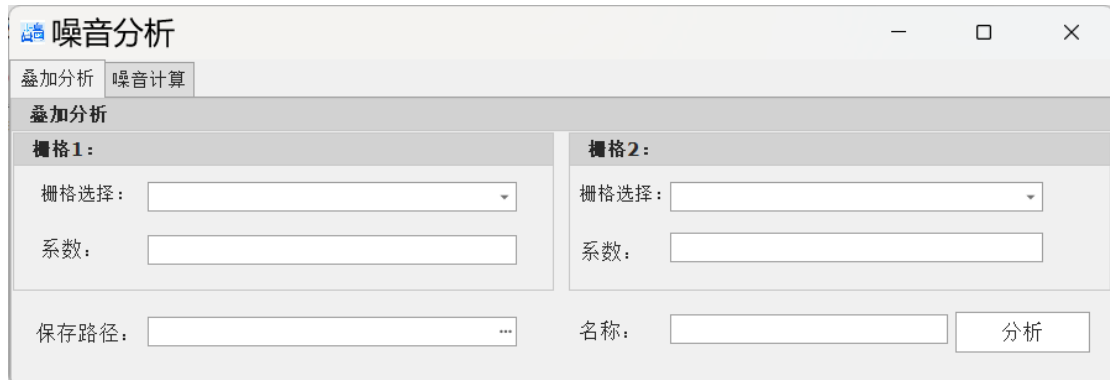
保存路径： ... 名称：

运行结果



5.6.2 叠加分析（地图代数）

5.6.2.1 界面



5.6.2.2 操作步骤

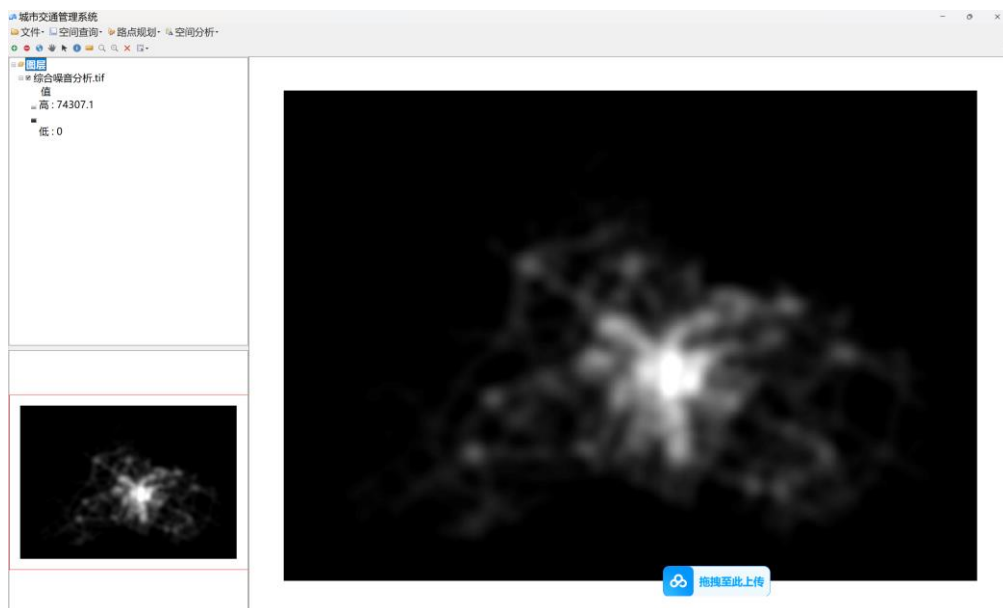
设置栅格→设置参数→设置路径和名称→分析

5.6.2.3 运行演示

设置参数与保存路径

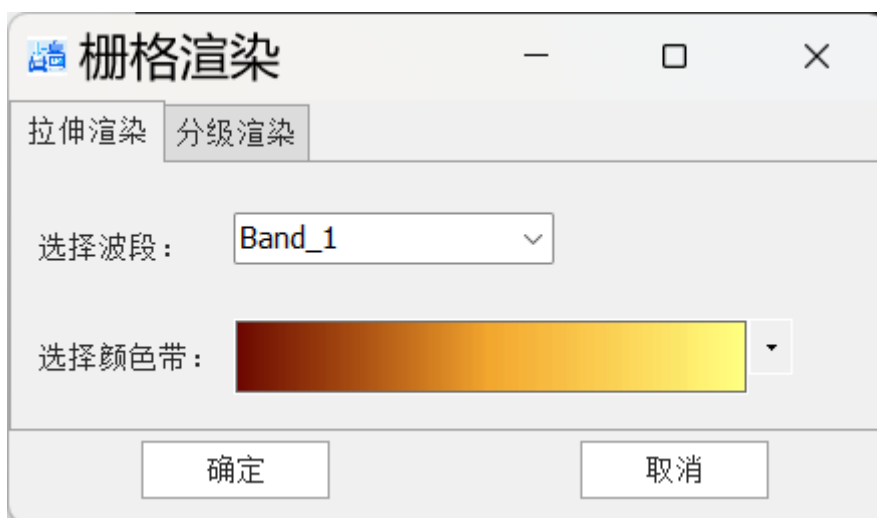


分析得出栅格图



5.6.3 栅格渲染

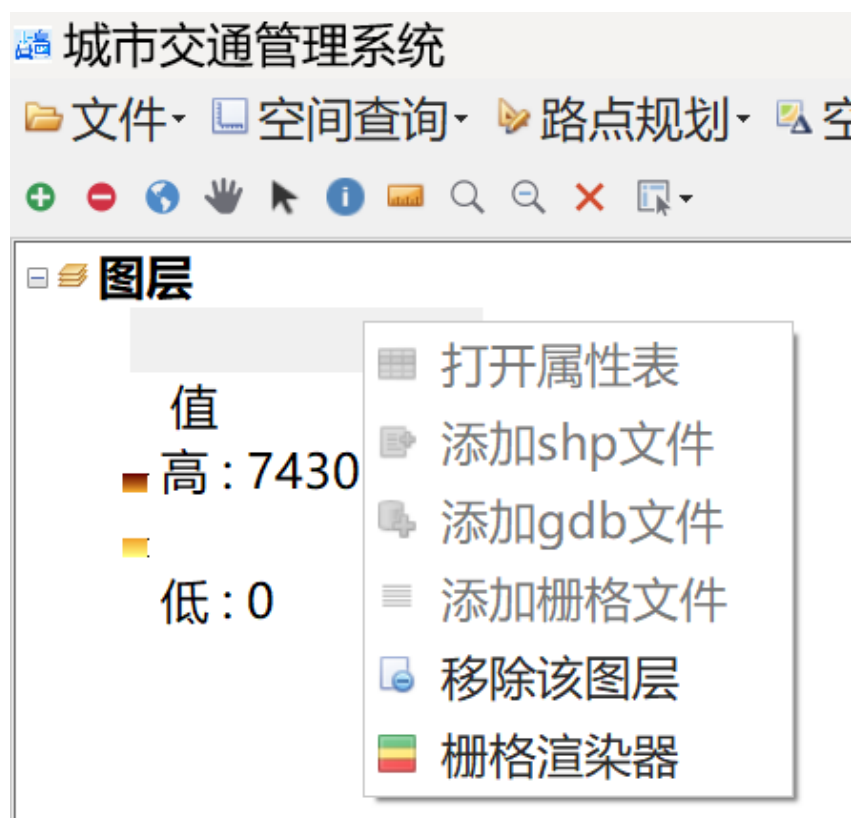
5.6.3.1 界面

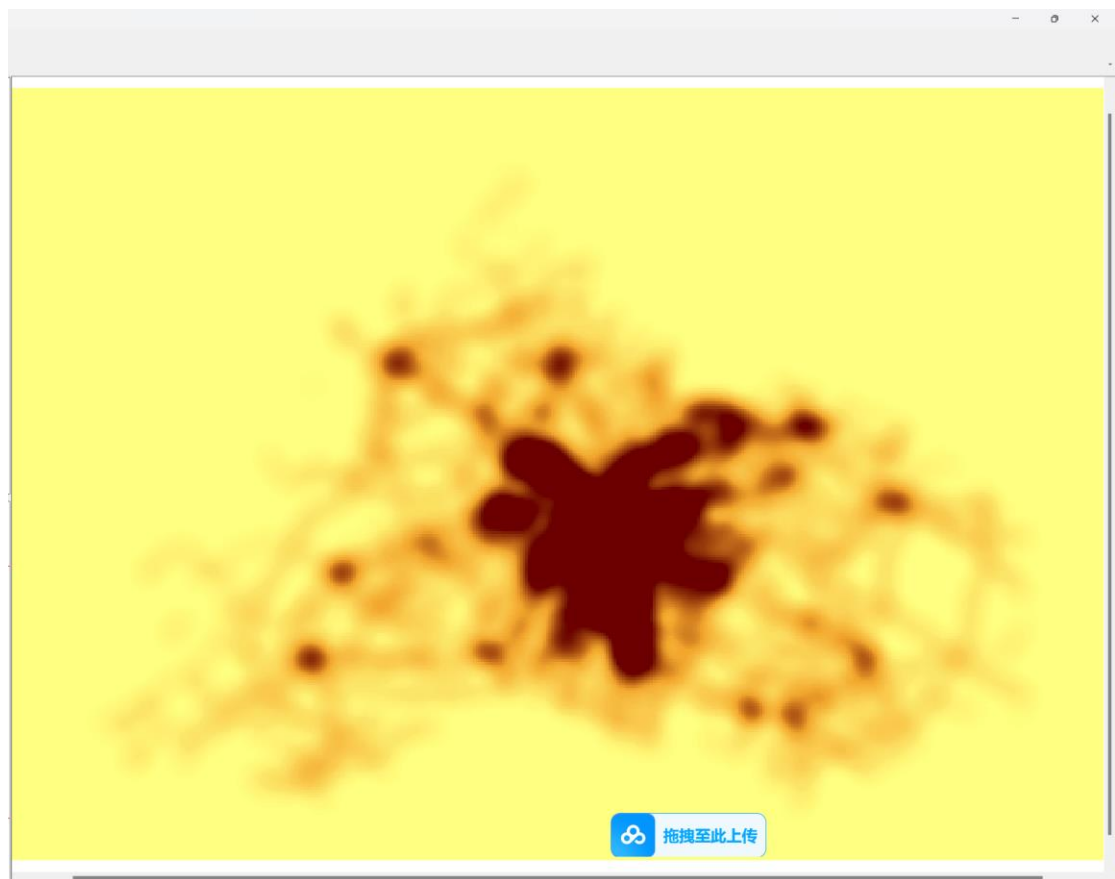


5.6.3.2 操作步骤

右键菜单→打开栅格渲染器→选择渲染类型→选择波段→选择色带

5.6.3.3 运行效果

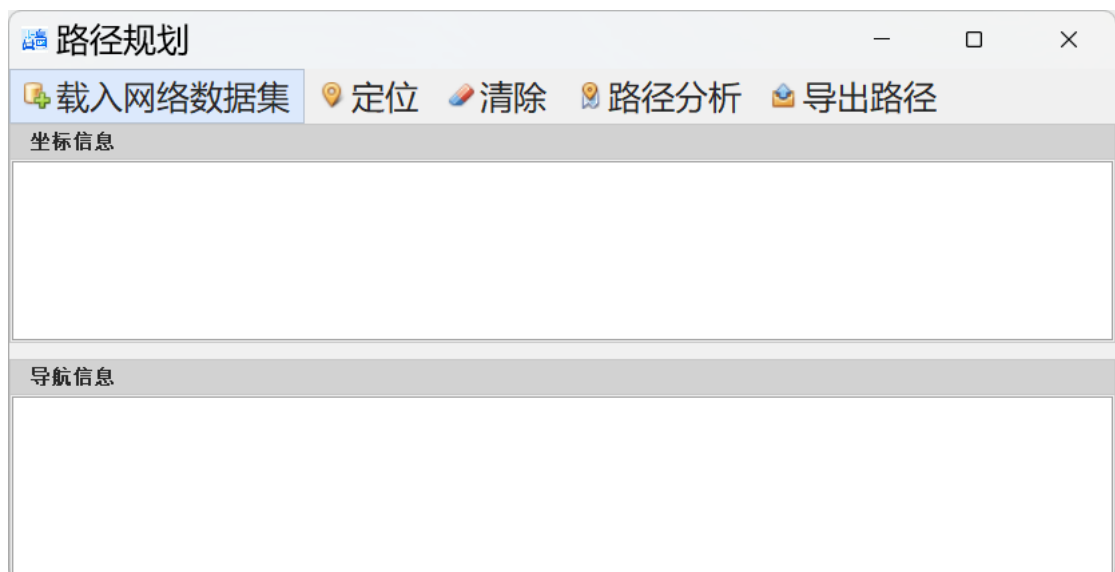




5.7 路径分析

5.7.1 最短路径生成

5.7.1.1 界面



5.7.1.2 操作步骤

载入网络数据集→定位→路径分析

5.7.1.3 运行效果

最短路径生成:



导航信息

路径规划
—
□
×

载入网络数据集
 定位
 清除
 路径分析
 导出路径

坐标信息

1129 , 833
293 , 108
88 , 800

导航信息

Turn right on 未知道路 Drive 0.2 mi ~ < 1 min

Turn left on 龙华东路 and immediately turn right on 车城西三路 Drive 0.2 mi ~ < 1 min

Continue on 天鹅西湖南路 Drive 60 ft ~ < 1 min

6 项目总结

6.1 系统测试

序号	功能分类	功能项	运行结果	问题描述
1	地图及图层管理	新建地图文件	正常	
2		打开地图文件	正常	
3		加载矢量图层	正常	
4		移除矢量图层	正常	
5		鹰眼地图	有缺陷	刷新速度慢
6		加载栅格图层	正常	...
7	属性表操作	查看属性表	正常	
8		编辑属性表	有缺陷	无法对不能编辑字段锁定（shape、Object）
9		删除属性表	有缺陷	删除设置只读的类型要素会抛出异常
10		导出属性表	正常	
11	空间查询	按属性查询	正常	
12		空间统计	正常	
13		站点查询	有缺陷	效率低，面对大量矢量数据运行速度慢
14	路点规划	创建要素类	有缺陷	不能将要素类自动添加进地图（需手动）
15		要素编辑器	正常	
16	空间分析	噪音计算	正常	
17		叠加分析	有缺陷	调用 python 脚本可能会无法退出
18		路径规划	正常	

6.2 项目总结

作为一名 GIS 本科生，GIS 软件的二次开发是我们必备的技能。“城市交通系统”项目开发，从整体来看，项目已基本完成了预设的总体目标和各项功能需求。

在项目开发过程中，收获颇丰。技术层面，深入学习并熟练运用了 ArcGIS Engine、C#、SQL Server 等多种开发工具和技术，提升了自己的编程和 GIS 应用能力；实践层面，通过实际参与项目，切实体会到从需求分析到最终实现的完整流程，了解到各环节之间的关联和影响，积累了宝贵的项目经验。

回顾项目开发过程，也有一些改进建议。一是在开发前期，应更加充分地进行技术调研和方案论证，以选择最优的技术路线和实现方案；二是在开发过程中，要进一步加强测试环节，及时发现并解决潜在的问题和漏洞，提高系统的稳定性和可靠性；三是可增加用户反馈渠道，便于后期根据用户实际使用情况进行功能的优化和扩展，使系统更贴合用户需求。总之，这次项目经历是一次宝贵的学习和成长机会，对未来的学习和工作都有着重要的指导意义。

7 附件

7.1 操作演示视频

7.2 项目源码

7.3 地图数据（不含卫星地图）

7.4 远程仓库链接：

<https://github.com/AritiYoshino/UrbanTransportionSystem.git>